程序设计综合实验

第 3 周：链表的方方面面（第一周：基础与 ASEP 应用）

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-09-30

第 3 周：链表的方方面面（第一周：基础与 ASEP 应用）

本讲范围与路线图

上周的位置与本讲主线

上周讲的是排序与分治：在**数组**这一种存储方式上把插入、冒泡、快速、归并四种方法实现出来，并从快速排序抽出分与治的一般做法。本讲不换要解决的问题，换的是**数据的表示**。

链表是一种线性数据结构：每个元素各自放在一个**节点**里，节点除了元素本身，还存一个指向下一个节点的**指针**，元素之间的次序完全由这些指针给出，节点在内存中的位置不必相邻。四个相关术语在这里先给出定义，图示与展开的讨论放在下一节：**节点**是存放元素的存储单元，由数据域与指针域两部分组成；**头节点**是链表中排在最前面的那个节点；**尾节点**是链表中最后一个节点，它的指针域为空；**空链表**是不含任何有效节点的链表。

表示一换，上周建立起来的代价直觉要重算：数组按序号取元素只要一次寻址，链表取第 k 个元素要从头节点出发走 k 步；数组在头部插入元素要让后面的元素整体腾位，链表做同一件事只改两个指针。本讲的内容，就是围绕这个差别展开的一套结构与一套操作。

环节	讲什么	为什么排在这里
动机与概念	数组的三条局限：内存要连续、长度固定、头部与中间插入删除要搬移元素；链表用分散存储加指针连接回避它们	先立存在理由，否则结构与操作只能靠记忆
结构与定义	节点（数据域与指针域）、头节点、尾节点、空链表	定义先立，后面每个操作都要用这套术语
存储与内存	分散存储多存一个指针的内存代价；C 的指针与 Julia 的对象引用在生命周期与空值上的差异	同一种结构在两种语言里从哪里分野，在这一段交代

环节	讲什么	为什么排在这里
代价对照与选型	两种表示在访问、插入、删除上的代价对照；按访问模式与更新模式立选型判据	上周的代价直觉换表示后必须重算，判据是本讲的落点
操作与 C 实现	遍历、按序号访问、按值查找、插入与删除的逻辑；节点定义与创建、插入与打印、删除的三种写法、哨兵思想、释放与内存泄漏	操作逻辑先于代码；这一段真正的难点是边界条件
Julia 实现	用抽象类型把空链表与非空链表分成两个类型（Nil 与 Cons）；链表操作的 Julia 写法	换一种语言看同一结构，检验“次序由指针给出”这条结论是否真的建立起来了
类型扩展	双链表、循环链表；静态链表用下标模拟指针	单链表讲透之后，变体只是代价上的取舍
链表上的算法	合并、拆分、交换、反转、检环、相交、倒数第 k 个；把上周的四种排序搬到链表上并重算代价	结在链表与上周分治的接续处：拆分长链与合并有序链正是分与治在链表上的对应物
常见错误与练习	边界条件为什么是链表代码麻烦的根源；课堂练习与课后作业	总结、要点与常见错误一律后置于练习，最早只能出现在示例之后

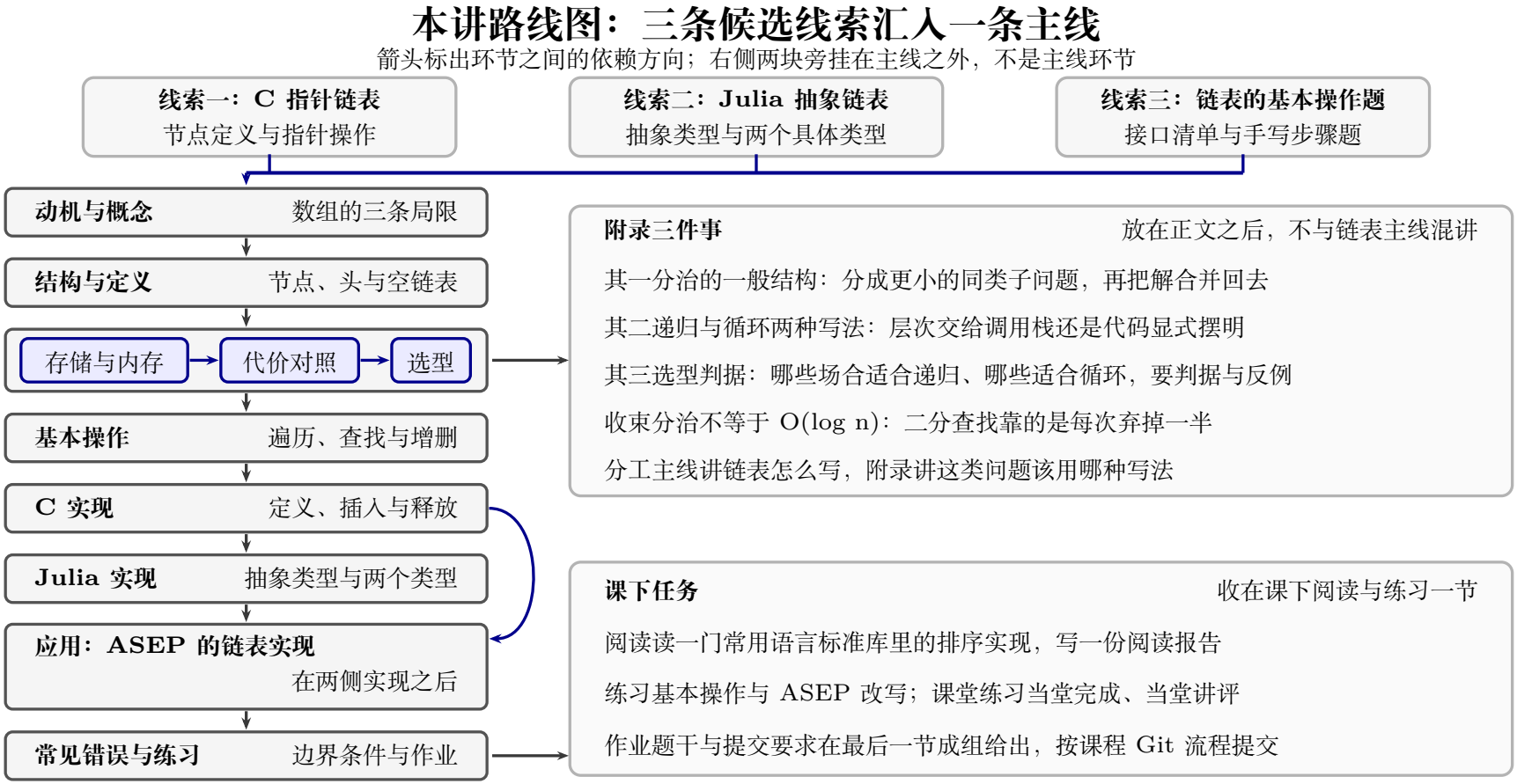


图 1: 本讲路线图

本讲是链表的第一讲：讲链表本身的基础，不排综合操作题（合并、三等分、交换、反转、检环、相交一类）与链表上的排序，那些留给下一讲；本讲收一个应用作为基础练习，即用链表实现 ASEP 的一维格点模型，全部操作只走链、不使用数组。

附录三件事：分治、递归与循环

链表主线走完之后，本讲加一个附录，把正文里反复使用但没有抽出来的三种思维收成一般形式。附录讨论三件事：第一，分治的一般结构：把原问题划成若干规模更小的同类子问题（分），再从最小子问题出发把解合并回去（治），交代这套结构成立的前提、分解终止的基线，以及两个可调参数（每次分几份、分到什么规模停）。第二，同一个算法在分治中的两种写法：递归写法把分与治写成一个函数的自我调用，层次的推进交给调用栈隐式保存；循环写法把分的层次显式化为外层循环、把治写成内层合并。第三，选型问题：哪些场合适合递归、哪些场合适合循环，给出判据与反例。

附录末尾回答一个被反复混淆的问题：分治不等于 $O(\log n)$。二分查找能做到 $O(\log n)$，靠的不是把区间分成两半，而是分完之后能根据比较结果把一半**弃**掉；把集合分成若干份再求和，每一份仍然要访问，复杂度不会低于 $O(n)$。递归与循环的差别，在数组上两者都写得出来、看不分明，换到链表上（删除与释放最容易撞到递归深度这一层）才暴露出来，所以这三件事放在链表讲完之后收。

语言与工具

本讲的示例与伪代码只用 **C 与 Julia** 两种语言，分工是：C 把内存与指针摆到明面上，找人要内存、还内存、空值约定都要自己交代；Julia 把类型与抽象摆到明面上，分配与回收交给运行时。同一个链表结构在两边写法上的差别（指针与对象引用、手动释放与自动回收、`NULL` 与 `nothing`），正是本讲要看清的东西。

C 的编译统一用 clang（示例都在 `clang -Wall -Wextra -g` 下编译运行过），调试统一用 lldb；调试器用法留到实验课，正文只在指出“空指针访问”这类错误时提到必要的调试动作。本讲所有可运行示例都放在 `l03-linked-list/examples/` 下，正文引用的输出与数字取自本机实跑，所用版本为 clang 19.1.7 与 julia 1.12.6；换一台机器后计时数字会变，量级与增长趋势不变。

本讲不在正文排练习：练习全部放在各节的 detail 里，含题干、参考实现与讲评要点。课堂练习当堂做，课后作业按课程 Git 流程提交，题干在最后一节成组给出。

从数组到链表：动机、结构、内存与选型

从数组到链表：动机、结构、内存与选型

上一周的排序全部做在数组上，数组是本课程到此为止唯一用过的存储方式。本讲换的不是要解决的问题，而是数据的表示：元素不再按地址顺序排在一起，而是各自散落在内存里，靠指针互相连接。换表示之前必须先回答一个问题，即链表解决了数组的什么困难、又为此付了什么代价。这一节把存在理由、结构定义、内存代价与选型判据依次立起来；后面几节的操作与实现，都建立在这一节的定义与代价账上。

为什么需要链表：数组的三条局限

数组（array）是一段连续的内存，存放同一类型的元素，元素的位置由首地址加上下标乘元素大小的偏移量算出。这条“地址能算出来”的性质是数组全部优点的来源：按下标访问任何元素都是 $O(1)$，没有额外的结构开销，连续排布还让相邻元素一起被带进高速缓存（CPU 与内存之间的一层小块快速存储，一次访存会把相邻数据一并搬进来）。

三条局限也从这条性质来，它们都是“连续”二字的直接后果。

1. **要求一整块连续空闲内存**：元素越多，要申请的连续区间越长。程序运行一段时间之后，内存被反复申请与释放切成碎片，空闲总量可能远大于数组所需，却找不出一块足够长的连续区间，于是大数组分配不出来。
2. **容量在分配时定死**：数组的长度一经分配就固定，要放更多元素，只能另外申请一块更大的连续区间，再把原有元素逐个复制过去，复制量正比于原长度，代价 $O(n)$。
3. **头部与中间插入、删除要搬移元素**：在数组的第 1 个位置插入一个元素，后面的每个元素都要向后挪一格；元素越多，挪得越多，代价 $O(n)$。删除同理，被删元素之后的元素要整体前移。

链表正是为回避这三件事而存在的：节点各自单独申请，因此对连续区间没有要求；插入一个元素才申请一个节点，因此不存在“扩容”；插入与删除只改动少数几个指针，因此不必搬移元素。

淘汰数组当然不是结论。数组三条优点的价值没有变，链表换来的只是那三类操作上的便宜；这笔便宜要连着代价一起算，见选型一节。

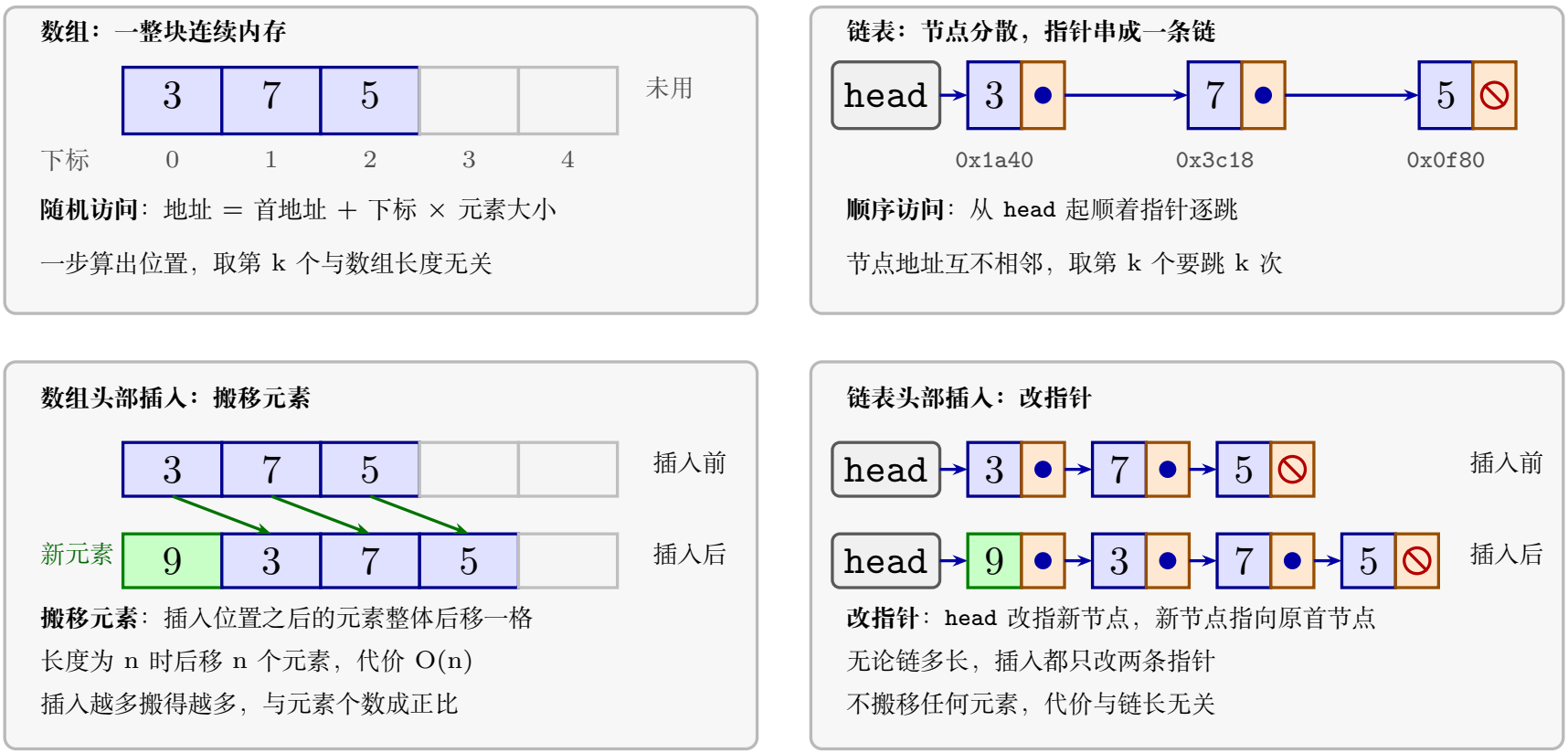


图 2: 数组与链表的存储与代价对照

Q 1 详解 1

链表的结构与定义：节点、头节点、尾节点与空值

链表（linked list）是一种线性数据结构：每个元素各自存放在一个节点里，节点之间用指针连接，元素的先后次序由这些指针给出。

四个术语在这里给出完整定义。

- **节点**（node）：链表的基本单元，由数据域与指针域两部分组成。数据域存放元素本身；指针域存放下一个节点的地址，在 C 里是指针，在 Julia 里是对象引用。
- **头节点**（head node）：链表中排在最前面的节点，它的地址保存在 head 变量中。head 是链表的入口，拿到 head 就能顺着指针走到链表的任何位置。
- **尾节点**（tail node）：链表中排在最后面的节点，它的指针域为空，表示后面没有节点了。
- **空链表**：没有任何有效节点的链表，此时 head 为空。

空值在两种语言里的记法不同：C 里是 `NULL`，Julia 里是 `nothing`。空值表达的是“后面没有节点”，它不是数据，读它的数据域是错误。

链表与数组的分水岭是一句话：**节点在内存中的位置不必相邻，元素的次序完全由指针给出**。数组的地址能由下标算出来，所以能随机访问；链表的节点位置与元素次序无关，只有指针知道下一个节点在哪里，取第 k 个元素必须从 head 出发走 $k-1$ 步，只能顺序访问。这条差别是后面所有代价差异的源头。

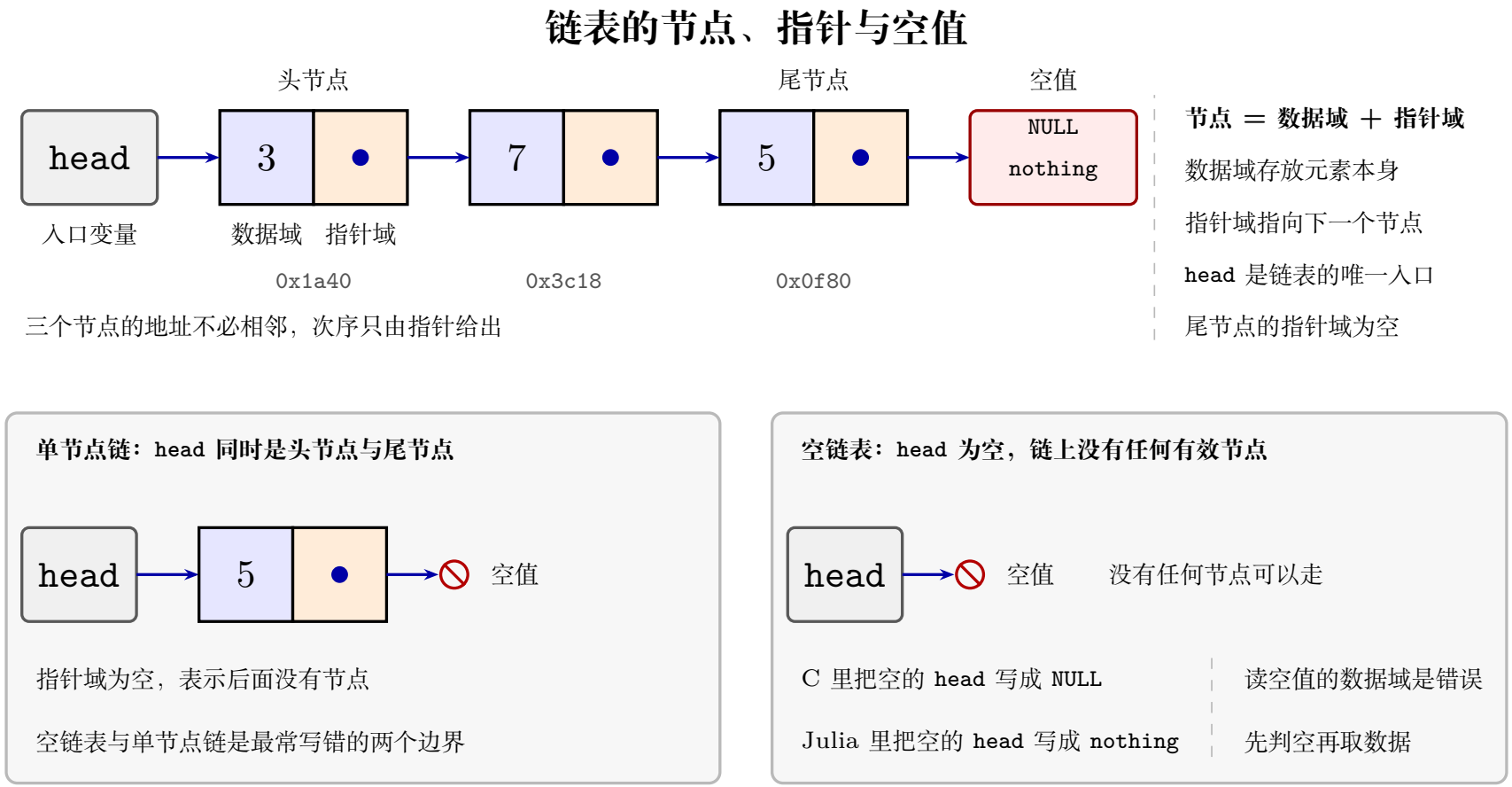


图 3: 链表节点的结构、指针与空值

Q 2 详解 2

存储表示、内存代价与指针／引用的差异

存储表示：节点分散在内存各处，head 保存第一个节点的地址，尾节点的指针域为空。数组与链表的区别可以这样记：数组的次序写在地址里，链表的次序写在指针里；地址是算出来的，指针是一格一格跟着走的。

内存代价：每个节点除数据域之外还要多存一个指针域，所以存同样多的元素，链表比数组多占内存。实跑数字（脚本：`examples/array_vs_list_cost.c` 与 `examples/array_vs_list_cost.jl`）：C 侧 `sizeof(int)` 是 4 字节、`sizeof(Node)` 是 16 字节（数据 4 加指针 8，多出的 4 字节是编译器为对齐插入的填充）；Julia 侧 `sizeof(Int)` 是 8 字节、`sizeof(Node)` 是 16 字节。存 500 万个整数，C 侧数组实测 20 MB、链表实测 80 MB，链表是数组的 4 倍。

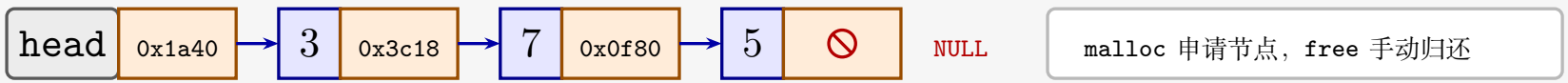
同一个 next 在两种语言里的两种含义：

- C 侧的 next 是**指针**，存放的就是一个内存地址。取地址与解引用都要程序自己写出来，节点的内存用 `malloc` 申请、用 `free` 归还，空值是 `NULL`；谁申请谁归还，编译器不替程序记账。
- Julia 侧的 next 是**对象引用**，地址不可见。节点由构造函数生成，回收交给垃圾回收（运行时自动找出不再被引用的对象并归还其内存），空值是 `nothing`；节点字段写成 `Union{Node,Nothing}`，把“要么指向一个节点、要么为空”写进类型，读之前必须先判空。

两侧的分工可以用一句话概括：C 交出对内存的完全控制，同时交出全部责任；Julia 用类型与回收器换掉内存泄漏与悬空指针两类错误，代价是释放时机不由程序指定。逐项对照见下方详解块。

同一链表的内存与空值：C 的指针与 Julia 的对象引用

C：指针域里写着真实地址，地址可见，空值是 NULL



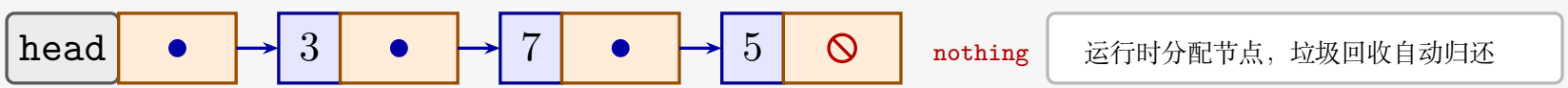
节点地址由 malloc 决定，三个节点在内存里互不相邻；

指针域里写的就是下一个节点的地址，谁申请谁归还。

对应关系：指针 → 对象引用；malloc / free → 运行时分配与垃圾回收；NULL → nothing

两处差别：地址可见还是不可见；释放由程序决定还是由回收器决定。

Julia：引用域里只是一个引用，地址不可见，空值是 nothing



指到哪个对象由运行时安排，引用域里看不到地址；

不需要写释放代码，也不会出现内存泄漏与悬空指针。

图 4: 同一链表在 C 与 Julia 中的内存与空值对照

详解 3

数组与链表的效率对照表

两种表示采用相反的存储策略，因此性质与操作代价成对出现：数组拿连续内存换随机访问，链表拿额外的指针与分散存储换插入删除的便宜。这张对照表是本节的骨架；它最危险的读法是把 $O(1)$ 与 $O(n)$ 当成无条件的结论，所以表后紧跟两条前提。

	数组	链表
存储方式	连续内存空间	分散内存空间
容量扩展	长度不可变	可灵活扩展
内存效率	元素占用内存少、但可能浪费空间	元素占用内存多
访问元素	$O(1)$	$O(n)$
添加元素	$O(n)$	$O(1)$
删除元素	$O(n)$	$O(1)$

- **链表的插入与删除是 $O(1)$，前提是已经持有要改的那一处指针**：头插要拿 head，中间插入与删除要拿前驱节点。如果只知道要处理第 k 个元素，从 head 走到第 $k-1$ 个节点这一步本身就是 $O(n)$，总代价仍是 $O(n)$。
- **数组的访问元素是 $O(1)$，前提是已经知道下标**：下标已知才能一次算出地址；按值查找要从头比一遍，代价 $O(n)$。

两条前提合起来得到本讲最要紧的一句话：**复杂度数字要连同前提一起读**，否则这张表会被读成“链表处处更快”。表里还藏着一件与常数有关的事实：$O(1)$ 与 $O(n)$ 只数操作次数，不问这一次操作是否命中缓存，所以实测倍数常常比复杂度表给人的印象更极端，也可能更缓和。表行术语的完整解释与实跑全表在下方 detail。

详解 4

选型判据：什么时候用数组，什么时候用链表

判据从本课程自己的三个场景里抽出来，不从清单上背。

- **场景一，随机行走的格子状态表**（实验二的数组版）：程序维护一张格子状态表与一张统计结果表，每走一步都要读当前格子的计数再加一，收尾时还要按下标读任意格子的结果做统计。这里的访问以随机访问与顺序扫描为主，格子数在开始前就定下来、不再变化。索引直接算出地址，数组是自然的选择。
- **场景二，同一个随机行走改用链表表示**（实验二检查点②）：把每个格子位置存成一个链表节点，功能与场景一完全相同。这是刻意安排的“同一功能换表示”：换成链表之后插入与删除变成改指针，但按下标取某个格子必须顺着链走，两种实现在访问、插入与内存占用上的差异要亲手量出来写进报告。
- **场景三，大自然数运算**（实验三）：一个大数的每一位存成一个节点，低位在前。做加法时从低位起逐位相加，每算出一位就接到结果链表的头部，进位还可能让结果比原来更长。这里的写操作集中在头部插入，位数事先不知道，每一位又都是刚算出来的新元素，链表正好对口。

从这三个场景抽出两条判据轴与一条附加条件：

判据轴	偏数组	偏链表
访问模式	按下标随机取、顺序扫描一遍、二分查找	顺着指针走一遍即可，或者操作时已经拿着前驱节点
更新模式与规模	更新集中在尾部、整体重排、批量替换，规模事先已知或稳定	头部或中间插入删除频繁，规模事先不可知或频繁变化
附加条件	要与库函数、矩阵运算配合，需要连续内存与缓存友好	元素本身很大、搬移一次的代价高，或需要节点地址稳定

三条边界提醒，第一条是本讲最要紧的一句。

1. **链表不是更快，而是在另一类操作上更便宜。** 链表把数组里“搬移元素”的代价换成了“找位置”的代价。有下标要取元素时，链表一定比数组慢；只有已经站在前驱节点上做插入删除，链表才快。上一节实跑全表的第二、三行正是这两个方向：头部插入链表赢、顺序求和数组赢，谁也不能抵消谁。
2. **数组的连续存储带来缓存优势，常数因子往往比复杂度表更好看。** 复杂度只数操作次数，不问这一次访存是否命中缓存。连续排布让一次访存顺带把相邻元素带进高速缓存，于是两个同样标着 $O(n)$ 的写法实测可以差一个数量级。判断性能时，复杂度决定趋势，常数决定当下。
3. **两者可以混用。** 用一个数组存数据、另用一个数组存每个元素“下一个”的下标，就用下标模拟了指针：重排时只改下标数组，不搬移数据。这种结构叫**静态链表**（游标链表），适用于不允许动态分配内存或需要重排大对象的场合，细节见下一讲的类型扩展一节。

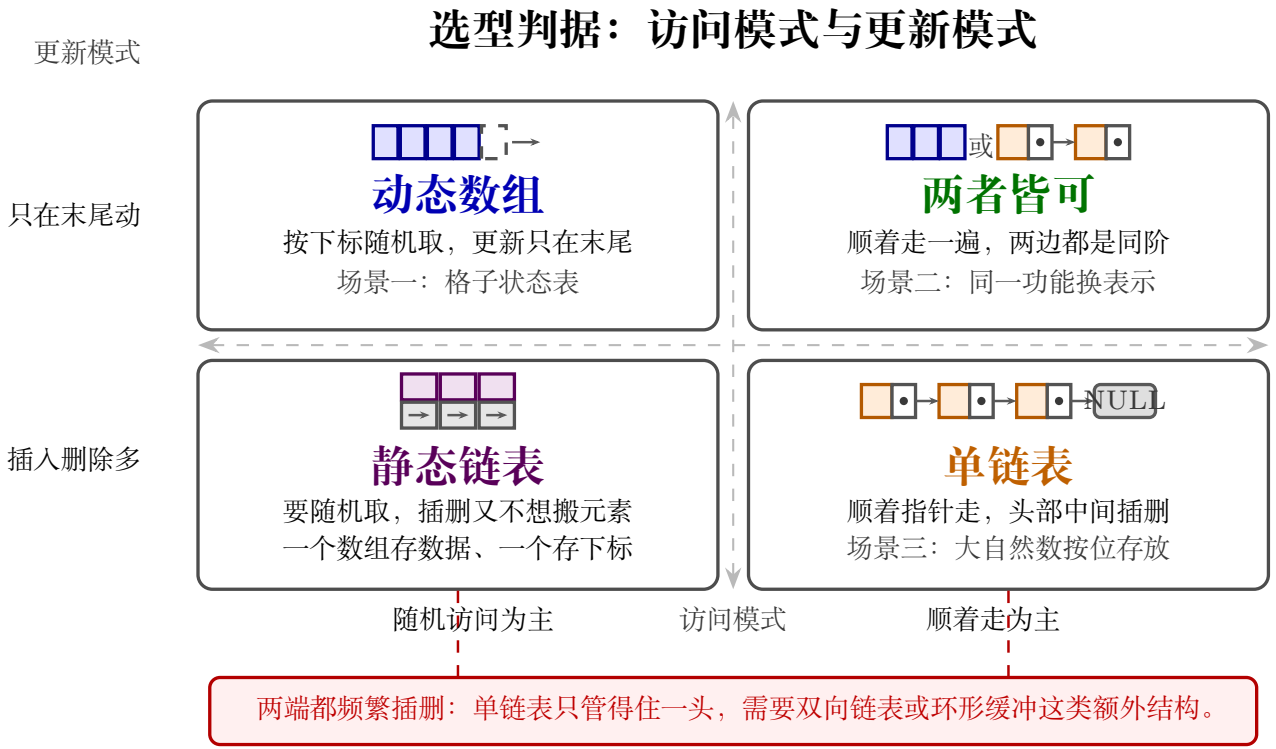


图 5: 数组与链表的选型判据：访问模式与更新模式

详解 5

单链表的基本操作

单链表的基本操作

上一节把链表的结构定下来了：一串节点，每个节点保存数据与指向下一节点的指针，头节点（head）是整个链表唯一的入口。结构只回答了节点在哪里，没有回答在它上面能做什么。本节讲单链表的四种基本操作：遍历、按序号访问与按值查找、插入、删除；它们是后面所有链表算法反复调用的原子动作。

四种操作按“定义与动机、示例、反例、最佳实践”四步走，重点落在三处最容易写错的地方：头插与尾插的代价差在哪里、删除之前为什么必须先拿到前驱、改指针的顺序为什么不能颠倒。下面引用的输出都是实跑结果，代码在 `examples/list_ops.c` 与 `examples/list_ops.jl`，反例在 `examples/bad_order.c`。

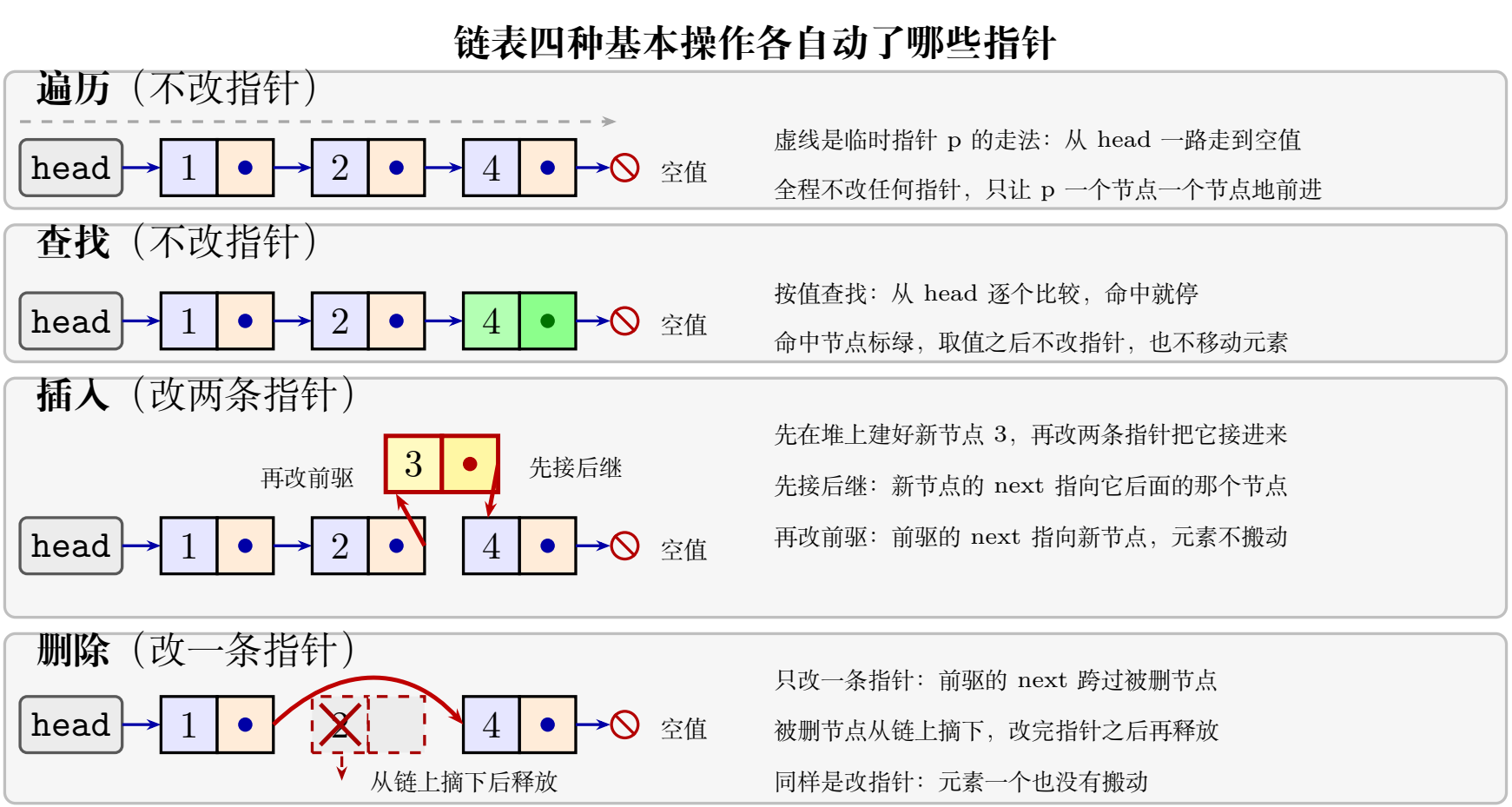


图 6: 单链表四种基本操作与指针改动

遍历：一切链表操作都要从 head 走一遍

定义：遍历（traverse）指从 head 出发取当前节点的数据，再沿当前节点的 next 前进到下一个节点，重复到空值为止。空值在 C 里是 NULL，在 Julia 里是 nothing。

动机：为什么遍历是链表一切操作的基础？因为链表除了 head 之外没有第二个入口。节点分散在内存各处，谁也不是靠下标就能算出来的。要在链表上做任何“定位”的事，找第 i 个节点、找值为 x 的节点、找尾节点、找某个节点的前驱，唯一的办法就是从 head 顺序走一遍。插入、删除、反转、合并、判环都以“先找到某个位置”为前提，遍历是这些操作共同的底座。

代价：按序号访问与按值查找都要从 head 顺序走，最坏情况走完整条链，代价 $O(n)$。这与数组的 $O(1)$ 随机访问正好相反：数组用首地址加下标一步算出地址，链表只能用“前进 i 次”换到第 i 个节点（本节下标都从 0 起）。**顺序访问**（sequential access）说的就是这件事：节点之间的次序完全由指针给出，只能一个接一个地取。

```c
// 遍历打印：沿 next 走到空值
void printList(const Node *head) {
    for (const Node *p = head; p != NULL; p = p->next) {
        printf("%d -> ", p->data);
    }
    printf("NULL\n");
}
```

```julia
# 遍历打印：沿 next 走到 nothing
function print_list(head::Union{Node,Nothing})
    p = head
    while p !== nothing
        print(p.data, " -> ")
        p = p.next
    end
    println("nothing")
end
```

实测输出（`./list_ops`，脚本先建出 `0 -> 1 -> 2 -> 3 -> 4`，这几行是原始输出的连续片段）：

```
尾插 4:                0 → 1 → 2 → 3 → 4 → NULL
  尾插走了 4 个节点
按序号访问 idx = 2:    data = 2
  访问走了 3 个节点
按值查找 target = 4:   下标 = 4
  查找走了 5 个节点
```

在 `0 → 1 → 2 → 3 → 4` 上取下标 2 的节点走了 3 个节点，查找值 4 走了 5 个节点：$O(n)$ 不是一个抽象记号，它就是这条链上一步一步走出来的步数。

Q 3 AI 1 详解 6 详解 7

插入：头插与尾插的代价差在哪里

定义：插入指在链表中增加一个新节点，做法是改指针而不搬元素。这与数组插入形成对照：数组在头部或中间插入要把后面的元素整体后移，代价 $O(n)$；链表只需要改动两条指针的指向。

头插（push front）：新节点的 next 指向原来的 head，再把 head 指向新节点。如果已经持有 head，这是两步操作，代价 $O(1)$，一个节点都不用访问。**尾插**（push back）：新节点要接在尾节点后面，问题是要先把尾节点找到，而单链表只有从 head 出发的路径，所以得沿 next 走完整条链，代价 $O(n)$。

中间插入：先找到前驱 prev，然后新节点的 next 接后继，prev 的 next 接新节点。找前驱本身要从 head 走，代价 $O(n)$；拿到前驱之后的改指针是 $O(1)$。

头插与尾插都只是改指针，两者的差别全在“改之前要不要先走一遍找位置”。这一点上是链表与自己比，而链表与数组比时，这条差异会反过来：数组的头插要把所有元素后移（$O(n)$），链表的头插是 $O(1)$；数组往末尾追加可以在末尾写（均摊 $O(1)$），链表不额外维护尾指针时反而要先走一遍（$O(n)$）。**头插便宜、尾插贵**是本节第一个反直觉的地方。

```c
// 头插：不遍历，O(1)
void pushFront(Node **head, int data) {
    Node *p = createNode(data);
    p->next = *head;
    *head = p;
}

// 尾插：先走到尾节点，O(n)
void pushBack(Node **head, int data) {
    Node *p = createNode(data);
```

```julia
# 头插：不遍历，返回新的头
push_front(head::Union{Node,Nothing}, data::Int) =
Node(data, head)

# 尾插：先走到尾节点
function push_back(head::Union{Node,Nothing},
data::Int)
    p = Node(data)
    head === nothing && return p
    tail = head
```

```c
    if (*head == NULL) {
        *head = p;
        return;
    }
    Node *tail = *head;
    while (tail->next != NULL) {
        tail = tail->next;
    }
    tail->next = p;
}

// 中间插入：已知前驱 prev，改两条指针
void insertAfter(Node *prev, int data) {
    Node *p = createNode(data);
    p->next = prev->next;        // 先接后继
    prev->next = p;              // 再改前驱
}
```

```julia
    while tail.next !== nothing
        tail = tail.next
    end
    tail.next = p
    return head
end

# 中间插入：已知前驱 prev，改两条指针
function insert_after(prev::Node, data::Int)
    p = Node(data, prev.next)    # 先接后继
    prev.next = p                # 再改前驱
    return p
end
```

实测输出（`./list_ops`，原始输出的连续片段）：

```
头插建表:              1 -> 2 -> 3 -> NULL
  头插 3 次共走了 0 个节点
头插 0:                0 -> 1 -> 2 -> 3 -> NULL
  头插走了 0 个节点
尾插 4:                0 -> 1 -> 2 -> 3 -> 4 -> NULL
  尾插走了 4 个节点
```

同一份输出稍后还有一行 `在 2 后面插入 25:  0 → 1 → 2 → 25 → 3 → 4 → NULL`。连续三次头插一共访问 0 个节点；在 5 个节点的链表上尾插一次访问 4 个节点（尾节点本身不必再前进）。这张实测账把 $O(1)$ 与 $O(n)$ 落成了两条能数出来的数。

Q 4 详解 8 详解 9

删除：为什么必须先拿到前驱

定义：删除指把一个节点从链表中摘掉（在 C 里还要释放它占用的内存）。与插入一样，删除只改指针，不搬元素。

动机：单链表每个节点保存的是指向后继的指针，被删节点自己并不知道它的前驱是谁。要把前驱的 next 跨过被删节点，就必须先拿到前驱；而拿前驱的唯一办法是从 head 顺序走一遍。这是单链表的一条固有代价，也正是双链表存在的理由之一：双链表每个节点多存一个指向前驱的指针，删除时不必再找前驱，代价是每个节点多一个指针的内存，以及插入与删除时要成对维护两条指针。

步骤（按值删除第一个等于 key 的节点）：先找到前驱 prev；把 prev 的 next 改成 victim 的 next，跨过被删节点；再释放 victim。三步的次序不能乱，下一小节专门讲这一点。

边界：如果被删的正好是头节点，它没有前驱，要改的是调用者的 head 本身。C 里这时形参要写成二重指针 `Node **head`，或者在链表头部预先放一个哨兵节点让删头与删中间在程序结构上等价。这两条路子在后面的 C 实现一节展开，本节先把这段边界点出来。

```c
void removeValue(Node **head, int key) {
    if (*head == NULL) {
        return;                          // 空链表
    }
    if ((*head)->data == key) {          // 删头：改的是调用者的 head
        Node *victim = *head;
        *head = victim->next;
        free(victim);
        return;
```

```julia
function remove_value(head::Union{Node,Nothing}, key::Int)
    head === nothing && return nothing
    head.data == key && return head.next   # 删头：改的是 head
    prev = head                            # 先拿到前驱
    while prev.next =/= nothing
        if prev.next.data == key
            victim = prev.next
```

```c
    }
    Node *prev = *head;                 // 先拿到前驱
    while (prev->next != NULL) {
        if (prev->next->data == key) {
            Node *victim = prev->next;
            prev->next = victim->next; // 跨过被删节点
            free(victim);               // 改完指针再释放
            return;
        }
        prev = prev->next;
    }
}
```

```ruby
            prev.next = victim.next         # 跨过被删节点
            return head                     # 内存交给垃圾回收
        end
        prev = prev.next
    end
    return head
end
```

实测输出（./list_ops）：

```
删除值 25:            0 -> 1 -> 2 -> 3 -> 4 -> NULL
  删除走了 3 个前驱候选节点
删除头节点 0:         1 -> 2 -> 3 -> 4 -> NULL
空表按序号访问 idx = 0: NULL
空表上删除:           什么也没发生
```

删值为 25 的节点时，先走了 3 个前驱候选才定位到它（与按值查找同一笔 $O(n)$ 账）；删头节点的分支一次都没走链，改的是 head。空链表上删除什么也不做，这就是边界要先处理的原因。

Q 5 详解 10

反例：改指针的顺序不能颠倒

链表代码真正麻烦的地方不在“不知道该改哪个指针”，而在“知道该改哪两条却改反了顺序”。两条指针里，凡是要用到的那个值只有一份拷贝，先覆盖它该去的地方就丢了。

反例一：中间插入的顺序颠倒。 正确顺序是先让新节点的 next 接上后继，再让前驱的 next 转向新节点。反过来的写法是：

```
void insertAfterBad(Node *prev, int data) {
    Node *p = createNode(data);
    prev->next = p;              // 前驱先指向新节点
    p->next = prev->next;        // 再取前驱的 next，取到的却是新节点自己
}
```

第二步里的 `prev->next` 已经在第一步被改写成了新节点自己，于是新节点的 next 指向自己，形成一个自环；原来后半条链（节点 3 与 4）在内存里还在，但再也没有任何指针能从 head 走到它们。实测输出：

```
顺序正确 插入前:  1 -> 2 -> 3 -> 4 -> NULL
顺序正确 插入后:  1 -> 2 -> 25 -> 3 -> 4 -> NULL
顺序颠倒 插入前:  1 -> 2 -> 3 -> 4 -> NULL
顺序颠倒 插入后:  1 -> 2 -> 25 -> 25 -> 25 -> 25 -> 25 -> 25 -> 25 -> 25 -> ... (打印到第 10 个节点仍未走到 NULL)
  新节点的 next 指向自己吗: 是 (成了自环)
  节点 3 还在链表里吗: 在，但已无法从 head 走到
```

注意这段输出里最要紧的一句是最后一行：节点 3 并没有消失，它还在内存里，只是从 head 出发再也走不到。这类错误在链表上叫**断链**，它既不会让程序立刻报错，也不会让编译器发出警告。

反例二：删除时把释放与取后继的顺序颠倒。 下面这段先 `free` 再取 `victim->next`：

```
Node *victim = k2;
free(victim);                  // 先释放
k1->next = victim->next;       // 再读 victim->next，读的已经是释放掉的内存
```

`free` 之后那块内存已经还给系统，`victim->next` 里现在是什么值，由系统决定。实测两种结果：不加检查器编译（`clang -Wall -Wextra -g`）时程序在打印中途直接段错误；加上地址检查器（`clang -fsanitize=address -g`）后，运行时报出确定性的 `heap-use-after-free`，指向读取 `victim->next` 的那一行。这条错误比反例一更隐蔽，因为它的后果取决于内存有没有被别处重用，同一份代码可能今天看起来对、明天崩。

一般规则：改指针之前先把所有还要用到的值保住，再覆盖。

- 插入时要保住的是后继的地址，它只存在 `prev->next` 这一份拷贝里，所以新节点的 next 必须先接上它。
- 删除时要保住的是 `victim->next`，被删节点一旦释放就取不到了，所以必须先把值取出来（或者先把前驱的 next 改完再释放）。

本节给这件事的口诀是**先接后断**：先把新节点的 next 接到该接的地方，再去断开或改写旧的那条指针。

Q 6 详解 11

最佳实践与操作一览

前面三处坑串起来就是四条做法。

1. **先画图再写指针。** 把改之前的形状与改之后的形状各画一遍，标出每个指针指向谁、哪一份地址只有一份拷贝。改指针的代码不长，出错几乎全在用脑内图代替纸面图的时候。
2. **边界先处理。** 空链表、只有一个节点的链表、删头、删尾这四种情形先写分支，再写主干。删头之所以麻烦，是因为它要改的不是某个节点的 next，而是 head 本身；这也是后面用哨兵节点把它消除掉的动机。
3. **改完就回读。** 每完成一次修改就遍历打印一遍，不要等整套算法写完再调。打印时给循环加上限，自环与断链会立刻现形。
4. **记住先接后断。** 插入时先接好后继，删除时先取出后继或先改完前驱。

四条做法收拢成一张操作一览表，本节讲过的操作、先决条件、改哪些指针与代价全在这里：

操作	先决条件	改哪些指针	代价
遍历	有 head	不改	$O(n)$
按序号访问	有 head	不改	$O(n)$
按值查找	有 head	不改	$O(n)$
头插	有 head	新节点的 next 指向原 head，head 指向新节点	$O(1)$
尾插（无尾指针）	有 head	先走到尾节点，尾节点的 next 指向新节点	$O(n)$
尾插（维护尾指针）	有 head 与 tail	尾节点的 next 指向新节点，tail 指向新节点	$O(1)$，代价是尾指针处处同步

操作	先决条件	改哪些指针	代价
中间插入	有前驱 prev	新节点的 next 接后继，prev 的 next 接新节点	$O(1)$，找前驱另算 $O(n)$
删除（非头）	有前驱 prev	prev 的 next 跨过被删节点，再释放被删节点	$O(1)$，找前驱另算 $O(n)$
删除头节点	有二重指针 head 或哨兵	head 指向第二个节点，再释放原头节点	$O(1)$，但要改 head 本身

表里最容易被误读的是最后三行的“另算”：链表的 $O(1)$ 插入与删除都以已经持有位置（前驱）为前提，找位置本身是 $O(n)$。

详解 12

C 实现：定义、插入、删除与释放

C 实现：定义、插入、删除与释放

本节把链表的结构、基本操作与选型判据落成可运行的 C 代码：编译用 clang（`clang -Wall -Wextra -g`），示例都在 `examples/` 下实跑过，正文引用的告警与输出抄自本机（clang 19.1.7）。调试用 lldb，本节只给一次检视示例，用法留到实验课。

C 与 Julia 的分工在这一节体现得最直接：

- C 侧：节点由 `malloc` 申请、由 `free` 归还，地址与生命周期都由程序自己管。好处是内存看得见，代价是本节后半段要处理的麻烦（内存泄漏、悬空指针、野指针）全都由此而来。
- Julia 侧：生命周期交给垃圾回收，麻烦就换成了另一类（类型与空值怎么表达），留到下一节。

C 侧链表：整体结构与由数组建链

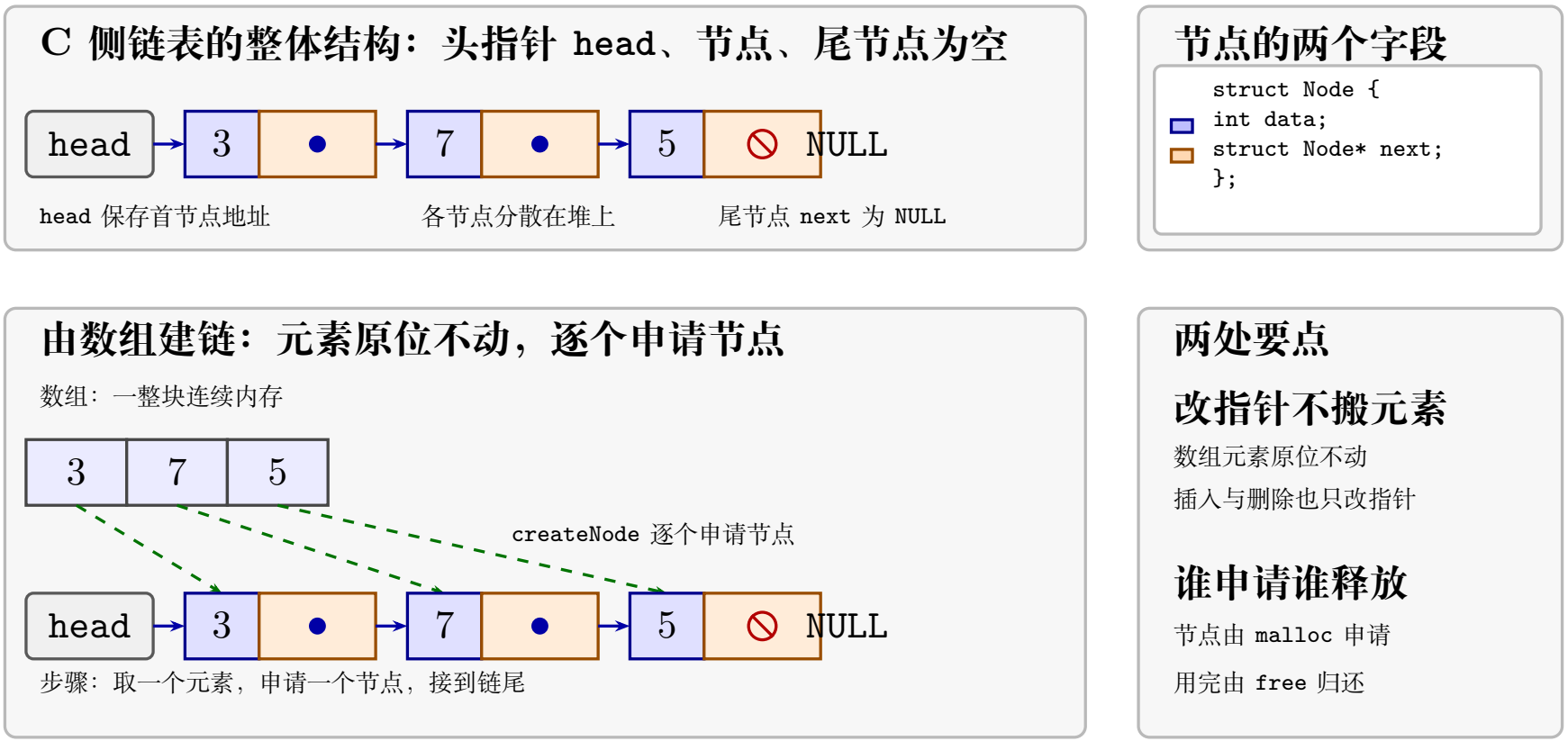


图 7: C 侧链表的整体结构与由数组建链

Q 7

节点定义、创建与初始化

先定术语。

- **节点**（node）：链表的存储单位，由两部分组成，**数据域**存元素本身，**指针域**存下一个节点的地址。指针域取值为 **NULL** 时表示后面不再有节点。
- **NULL**：C 里表示“不指向任何有效对象”的空指针常量（定义在 `<stddef.h>`，随 `<stdio.h>` 与 `<stdlib.h>` 一起可用）。
- **头节点**：链表第一个节点的地址所存放的位置。本课默认链表**带头节点**，且头节点是不存有意义数据的**哑节点**（dummy node，也叫哨兵节点）。
- **空链表**：只有哑节点、没有任何数据节点的状态。

节点定义统一取 `typedef` 写法：结构体标签与类型名合一之后，函数原型里不必再写 `struct` 关键字，将来成对写 `list.h` / `list.c` 时可以直接用。

```
// 定义链表节点结构
typedef struct Node {
  int data;
  struct Node* next;
} Node;
```

自引用的 `struct Node* next` 是关键：结构体在定义内部只能以指针形式引用自己，因为此时 `sizeof(Node)` 还未确定。指针域的类型是“指向 Node 的指针”，不是 Node 本身，链表能无限延长而结构体大小固定，全部理由都在这一行。

由节点到节点，分两步：创建单个节点，再初始化一条链表。

```c
// 用指针的方式创建一个新的节点
Node* createNode(int data) {
  Node* newNode = (Node*)malloc(sizeof(Node));
  if (newNode == NULL) {
    fprintf(stderr, "Memory allocation failed\n");
    exit(1);
  }
  newNode->data = data;
  newNode->next = NULL;
  return newNode;
}
```

```c
Node* initLinkedList(void) {
  // 创建头节点
  Node* head = createNode(0); // 示例：头节点数据为 0
  return head;
}
```

`malloc` 的返回值必须检查：申请失败时它返回 NULL，此后对 NULL 解引用就是崩溃。创建节点的四步顺序不是随意的：

1. 申请内存；
2. 检查申请是否成功；
3. 填数据域；
4. 指针域显式置空。

第 4 步尤其不能省：`malloc` 分配到的内存里是上一次使用留下的不确定值，不清零就等于埋一个野指针。

三个必须当场回答的问题：

1. 为什么需要头节点？头节点的数据域可以放什么？
2. 为什么创建节点时要使用 `malloc`，而不直接声明一个 `Node` 变量？
3. 为什么 `createNode` 返回指针，而不是返回 `Node` 类型本身？

下面的 detail 里给出这三个问题的思路，另附完整源码、编译与运行的全部输出，以及命名混用的后果。

详解 13

插入、数组转链表与遍历打印

链表只有两种“改结构”的动作：把某个 `next` 指向新节点，或者把某个 `next` 跨过被删节点。插入先看最简单的一种，**尾插**。

```
void insertNode(Node* head, int data) {
  Node* current = head;
  while (current->next != NULL) {
    current = current->next;
  }
  current->next = createNode(data);
}
```

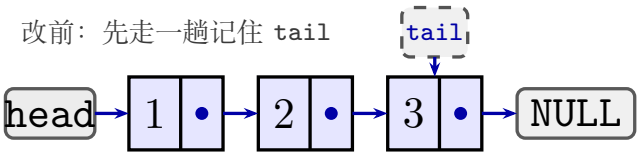
注意这里已经藏着一个代价：`insertNode` 必须从 `head` 一路走到尾，所以**一次尾插是 O(n)，不是 O(1)**。链表的插入本身只改一个指针，但“找到插入位置”这件事在单链表上无法省掉，没有存下尾指针就只能走一遍。连续尾插 n 个元素因此是 $O(n^2)$。

尾插：多存一个尾指针，省的是一整遍遍历

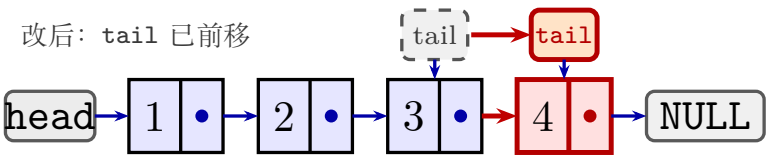
红色：改动　虚线：走链路径

改法一：存下尾指针

改前：先走一趟记住 tail



改后：tail 已前移



两条要改：1) 尾节点的 next 指向新节点；

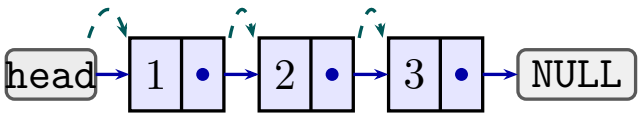
2) tail 前移到新节点。

少做 2)：下一次尾插又把同一条 next 改一遍，

刚接上的节点就丢在链外。

改结构只动第 1 条；第 2 条改的是 tail 自己。

改法二：每次都从头走到尾



0 步　1 步　2 步　3 步

链上已有 3 个节点时，走到尾节点要走 3 步。

第 k 次尾插走 k-1 步；n 次合计

0+1+⋯+(n-1) = n(n-1)/2 步（n = 5 时 10 步）；

存下尾节点之后只走 n-1 步记住它，

之后每次只接一步（n = 5 时只走 4 步）。

两版输出一样，差别只在每次都要走的这一遍。

要点：多存一个尾指针，省的是一整遍遍历。

走到尾节点这一趟省不掉，只是有尾指针时只走一次。

连续尾插 n 个的走链代价：n(n-1)/2（即 $O(n^2)$）与 n-1（即 O(n)）。

图 8: 存下尾指针：改两条指针，省一趟遍历

由数组批量建链时，这个代价会被放大。下面是一个反例，先别往下看，想想这个函数有什么问题：

```
1 // 向链表中批量插入新节点，不修改 head 节点
2 void insertNodes(Node* head, int* data, int size) {
```

```
  Node* current = head;
  for (int i = 0; i < size; i++) {
    insertNode(head, data[i]);
  }
}
```

```
void insertNodes(Node* head, int* data, int size) {
  Node* tail = head;
  while (tail->next != NULL) {
    tail = tail->next;
  }
  for (int i = 0; i < size; i++) {
    tail->next = createNode(data[i]);
    tail = tail->next;
  }
}
```

两版的输出完全一样，所以**看结果是看不出问题的**：差别在编译器的告警里，也在代价里。**一个声明了却用不上的中间变量，往往是理解不到位的痕迹，而不是无所谓的小事**，这一点在 `-Wall -Wextra` 下会被编译器直接点出来。

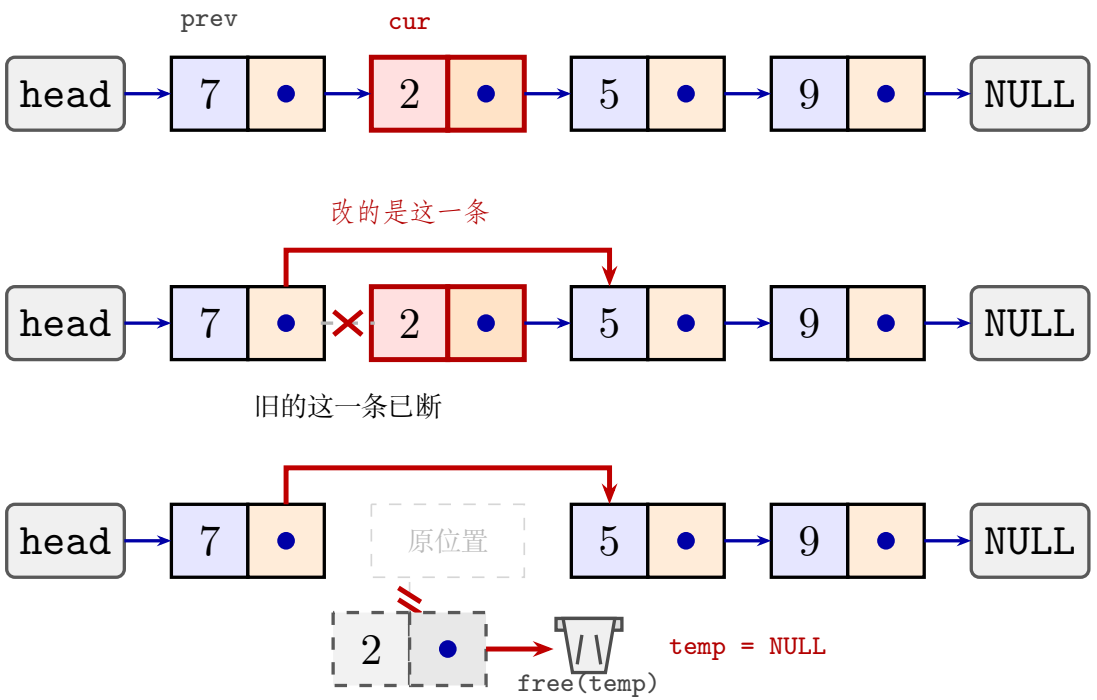
遍历是链表唯一走完全程的方式：

```
void printLinkedList(Node* head) {
  Node* current = head;
  while (current != NULL) {
    printf("%d -> ", current->data);
```

```
    current = current->next;
  }
  printf("NULL\n");
}
```

删除一个节点：定位、改指针、释放

红色：本步动过的地方



① 定位：两个临时指针

`prev` 停在目标的前驱上，

`cur` 停在要删的那个节点上。

② 改指针：让前驱跨过目标

```
prev->next = cur->next;
```

目标已不在链上，内存还没归还。

③ 释放：归还摘下来的节点

```
free(temp);
```

内存归还后，那个地址不得再用。

要点：改指针与释放是两件事，先让前驱跨过目标（把节点从链上摘下来），再归还摘下来的节点；释放之后的地址不得再访问。

图 9: 删除一个节点的三步：定位、改指针、释放

输出形如 `1 → 2 → 3 → NULL`。两个写法上的要点：

- 循环变量必须是 `current` 而不是 `head`，否则遍历完就再也找不到链表头了。

- 结束条件是 `current ≠ NULL`，走到尾节点的下一步才停，所以尾节点的数据会被打印，而 NULL 本身只作为结束标记被打印出来（这就是末尾那个 `NULL` 的来源）。

为什么要遍历链表，打印只是表象。真正的动机是前面讲过的顺序访问性质：链表没有下标，按序号访问、按值查找、求长度、删除指定值、求和求平均，全都必须先有一条能走得完整的遍历路径。遍历是链表一切操作的基础设施，其他操作都是“在遍历过程中做点别的事”。

两个必须当场回答的问题：

1. 反例里 `current` 声明了却没有使用，这个函数的问题出在哪几层？
2. 为什么要遍历链表？如果把 `printLinkedList` 里的 `printf` 换成一个计数器，这个函数还能做什么？

详解 14 AI 2

删除的三种写法：双指针、哨兵与递归

同一件事：从链表里删除**所有**等于 key 的节点。三种写法都能做完，差别在边界条件处理、形参形状与内存代价上。

写法一：双指针法（前驱与当前节点一起前进）

```
1  void deleteNodes(Node** head, int key) {
2    Node* current = *head, * prev = NULL; // 不合适但合法
3    while (current ≠ NULL) {
4      if (current→data == key) {
5        if (prev == NULL) {
6          *head = current→next; // 修改头节点
7        } else {
8          prev→next = current→next; // 跳过要删除的节点
9        }
10       Node* temp = current; // 这 3 行保证内存不泄漏
11       current = current→next;
12       free(temp);
13     } else {
14       prev = current;
15       current = current→next;
16     }
17   }
18 }
```

三个关键判断：

- **形参必须是二重指针** `Node** head`：函数要把“头节点”从旧地址改成新地址，改的是调用者手里的那个变量，所以必须拿到它的地址。
- **`free` 之前的三行顺序不能换**：先用 `temp` 记住要删的节点，再把 `current` 推到后继，最后才 `free`；顺序颠倒就再也找不到后继，后面整条链全部泄漏。
- **`prev == NULL` 这一支就是删头特判**：它只在“被删的节点恰好是头”时发生。

写法二：哨兵法（在头部临时加一个哑节点，把删头变成删中间）

```
Node* deleteNodesWithDummy(Node* head, int key) {
  Node dummy;
  dummy.next = head;
  Node* current = &dummy;
  while (current->next != NULL) {
    if (current->next->data == key) {
      Node* temp = current->next;
      current->next = current->next->next;
      free(temp);
    } else {
      current = current->next;
    }
  }
  return dummy.next;
}
```

这里的 `dummy` 是**栈上的局部变量**，不占堆、不需要 `free`；函数返回的是 `dummy.next`，也就是删完之后真正的头。

写法三：递归法（删完之后把子链表的头接回来）

```c
Node* deleteNodesRecursive(Node* head, int key) {
  if (head == NULL) {
    return NULL;
  }
  if (head->data == key) {
    Node* temp = head;
    head = head->next;
    free(temp);
    return deleteNodesRecursive(head, key);
  }
  head->next = deleteNodesRecursive(head->next, key);
  return head;
}
```

代价结论：头节点改变发生在 `head->data == key` 那一支，而返回上层的那一步会自动把新的头交出去，所以这个函数**不需要二重指针**；但每处理一个节点就压一层栈，**栈深正比于链表长度**。

详解 15 三种写法的完整代码与实跑输出在 `examples/list_delete.c`。

详解 16

三种写法的取舍：边界条件是麻烦的根源

三份代码摆在一起，能看出一个比“哪种写法更快”更要紧的结论。

麻烦的根源是删头节点这一条边界。 链表的第一个节点没有前驱，任何“改前驱的 next”的操作对它都失效。链表代码里七成以上的绕，绕的都是这一处：双指针法里那个 `prev == NULL` 分支、哨兵法里那个多余的 `dummy`、递归法里那个“返回值就是新头”的约定，全是为它准备的三种形状。

三者的取舍：

- **双指针法**：不动链表结构、不多占一个节点、不需要调用者接收返回值，接口最“朴素”；代价是形参变成二重指针，且函数体里始终有一条只在边界执行的支路。适合写进函数库时对外接口要稳定的场合。
- **哨兵法**：删头与删中间用同一段代码，边界被“消灭”而不是“处理”；形参是一重指针，函数体只有一条主路径。代价是恒多一个节点（本课示例把哑节点放在栈上，所以不占堆、不需要释放），且调用者必须把返回值赋回 head，漏掉这一句就等于什么都没删。
- **递归法**：代码最短，边界条件的处理被“交回上一层”这个动作顺带完成了；代价是栈深 $O(n)$，链表一长就会爆栈，而且每一步都要做一次函数调用，常数因子比另两者大。

选择的标准：

- 链表长度可达数百万级、或者要写成函数库的正式接口，取哨兵法或双指针法。
- 链表长度有限（几千以内）、或者正在用递归思路推导算法（比如链表上的归并排序），取递归法。

三者都不是错的做法，写代码时能说清自己选了哪一种、为什么不选另一种，比记住“哪一种最好”有用得多。

递归法与附录里“递归与循环”一节直接衔接：链表是自相似结构（删掉头之后的剩余部分仍是一条链表），所以它天然适合递归；生产代码里链表往往很长，所以工程上又常常把递归改写成迭代。**结构自相似的地方适合递归，数据规模未知或很大的地方适合循环**，链表正好同时具备这两种特征，也就成了这个矛盾最典型的例子。

详解 17

哨兵思想与链表结构体：从特判到等价

把两种删除写法的具体差别抽成一般形式，可以得到一个在数据结构的代码里反复出现的思维手法：**等价化**，用一个无害的额外对象，把一种特殊情形化成一般情形，从而消除代码里的特判。**一条只在边界执行、却与主路径语义相同的支路，是错误的高发地**，因为它平时不被测试覆盖，出问题也只在极端输入上暴露。

它的适用条件与代价：

- 收益：特判被消灭，主代码只剩一条路径，测试覆盖与阅读负担同时下降。
- 代价：结构里恒多一个多余对象。对本课的链表就是“链表恒多一个节点，有意义的数据总从第二个节点开始”，任何按位置计数的代码（第 i 个节点、长度、打印）都要记得这件事。
- 关键判断：代价是一次性的（初始化时付一次）、局部的（只在涉及头部时注意），而收益是**在每一次函数体里持续兑现**的。所以这类替换几乎总是划算的。

推演链条是三步：看清痛点（双指针法的 `if (prev == NULL)` 支路与主路径语义相同却写法不同）；问一句“能不能让它不存在”（如果头节点也有一个前驱，删头就变成删中间）；得到一般形式（在头部前置一个不存有意义数据的节点，它就是“原 head 的前驱”）。

哨兵法用的是这个手法的**临时版本**：哑节点在栈上，只在一次函数调用内存在，函数返回就“撤销”了等价化。如果一条链表的每个操作都怕删头，那就把这个哑节点**固化下来**：`initLinkedList` 直接返回一个哑节点，此后所有操作都默认“头节点是哑节点”。

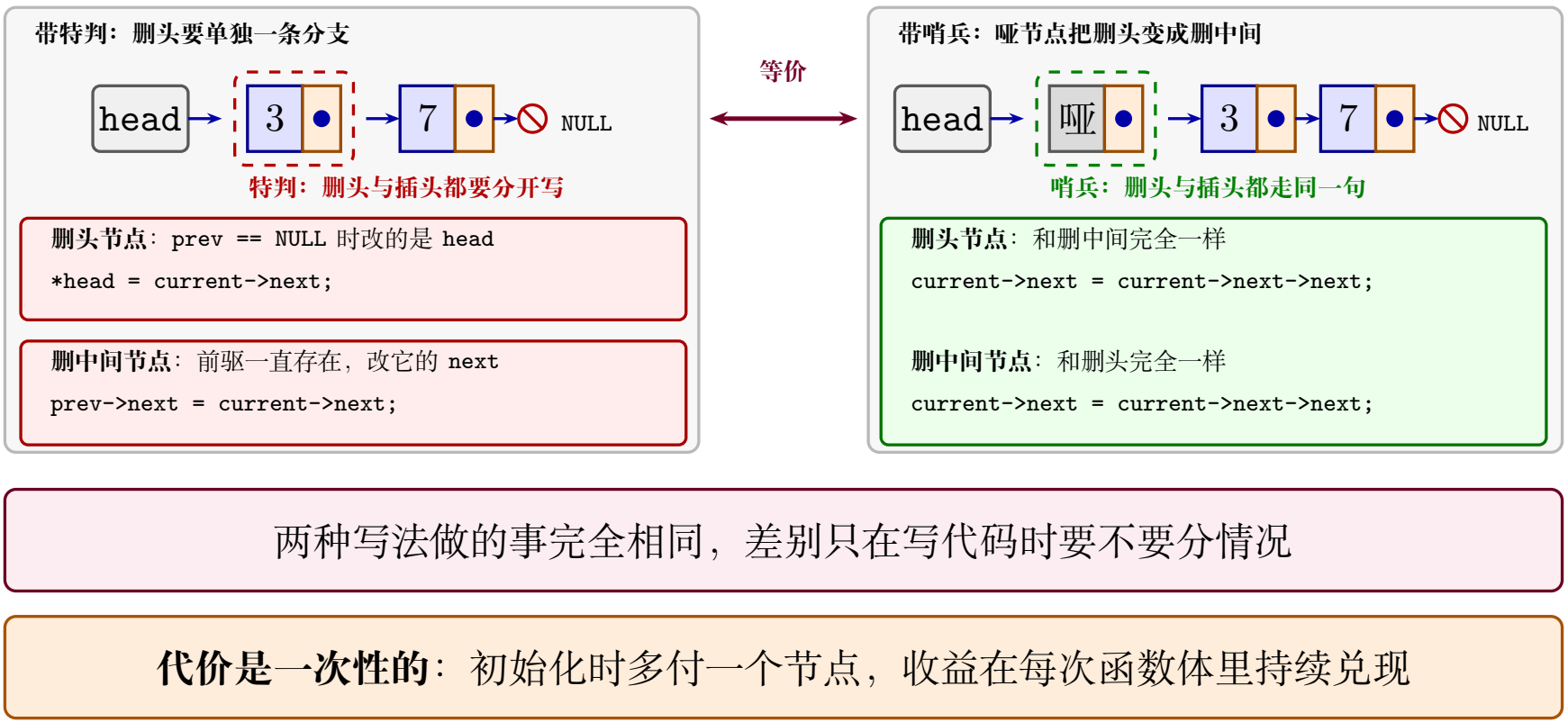


图 10：哨兵等价化：两种写法是同一件事

顺着这个思路再走一步，就得到**链表结构体**：

```
typedef struct LinkedList {
  Node* head;
```

```
} LinkedList;
```

这个结构体只有一个成员，看上去什么都没做，但它把“链表”与“节点”变成了两个概念：

- 函数形参写 `LinkedList* list`，读者立刻知道这个函数处理的是整条链。
- `list->head` 里装的默认就是哑节点，不必再在函数体里临时造一个。

这就是**对象思维**：把描述同一件事的数据与操作收进一个层次里，接口只暴露这一层，内部实现可以改（换带头节点的处理、加尾指针、加长度字段）而不影响调用者。本讲最后要把链表写进 C 函数库，前提就是先有这个层次。

三处容易走偏：

- **把哑节点当数据节点**：打印、求长度、按位置访问都必须从 `head->next` 起算。
- **以为哨兵法完全免费**：临时版本要调用者接收返回值（`head = deleteNodesWithDummy(head, key)`），漏掉这一句删除就白做；固化的哑节点把这个负担也去掉了，因为头节点永远不会被删。
- **把不变量写成约定**：一旦决定“链表带头节点”，它就成了一条贯穿全讲的不变量。后面的代码（插入、删除、释放、排序）都必须按同一条不变量写，否则就会出现“有的函数认为第一个节点是数据、有的认为是哑节点”这种最难查的错误。链表的调试成本大半来自不变量不统一，而不是来自指针本身难写。

释放链表与内存泄漏

`malloc` 申请的内存只有 `free` 能归还。**内存泄漏**指程序丢失了对一块已申请内存的引用，导致它既不能被使用、也不能被归还。

- 进程退出时操作系统会回收全部内存，所以短命程序里的泄漏看不出来。
- 循环里反复建链的测试程序、长期运行的服务，泄漏会一层层累积到把可用内存吃光。

这个后果可以当场量出来。本节第一个示例 `examples/list_basic.c` 建了一条 4 个数据节点的链表而没有释放它，用 clang 的 AddressSanitizer 跑一遍：

```
$ clang -Wall -Wextra -g -fsanitize=address list_basic.c -o list_basic_asan
$ ./list_basic_asan
...
==1342818==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 16 byte(s) in 1 object(s) allocated from:
    #1 0x5623996fd8a4 in createNode list_basic.c:18:26
    #2 0x5623996fd98a in initLinkedList list_basic.c:30:10
    #3 0x5623996fdd26 in main list_basic.c:67:16

Indirect leak of 48 byte(s) in 3 object(s) allocated from:
    #1 0x5623996fd8a4 in createNode list_basic.c:18:26
    #2 0x5623996fdb24 in insertNodes list_basic.c:49:18

Indirect leak of 16 byte(s) in 1 object(s) allocated from:
```

```
    #1 0x5623996fd8a4 in createNode list_basic.c:18:26
    #2 0x5623996fda09 in insertNode list_basic.c:39:19

SUMMARY: AddressSanitizer: 80 byte(s) leaked in 5 allocation(s).
```

节点是 16 字节（4+8 加对齐填充），5 个节点 80 字节，报告里的数字与节点个数完全对上，并把每一块内存**是谁申请的**指了出来。这就是“不释放”长什么样：程序正常退出、输出一切正常、报告才说出来。

释放的两条纪律：

1. **先记住后继，再 free 当前节点**。free 之后那块内存不能再读，current→next 也就不能读，所以必须在 free 之前把后继存进临时变量。
2. **结束时把调用者的 head 置空**。free 只归还内存，不会改变任何指针变量的值；调用者的 head 仍然保存着那个已经失效的地址，就成了悬空指针。置空只花一行，却把“这块地址还能不能用”变成可判定的事实。

```c
void freeList(Node** head) {
  Node* current = *head;
  Node* temp;
  while (current != NULL) {
    temp = current->next;
    free(current);
    current = temp;
  }
  *head = NULL; // 将调用者的头指针置为 NULL
}
```

形参用二重指针的原因与删除函数一样：函数要改的是调用者手里的 head 变量。

另有一版更简洁的写法，它把“推进”与“置空”两件事合并到了一个二重指针上：

```
1 void freeListSimple(Node** head) {
2   while (*head ≠ NULL) {
3     Node* temp = *head;
4     *head = (*head)->next;
5     free(temp);
6   }
7 }
```

这个写法逻辑正确：循环条件结束时 *head 恰好为 NULL，所以调用者的 head 也被置空了，本课实测确认了这一点。它不推荐的理由不在正确性，而在可读性与调试：每释放一个节点就改写一次调用者的 head，读代码的人必须同时在脑里跟踪两层指针；用 lldb 观察时 *head 的值每一轮都在变，“把链表头释放掉”这件事反而更难看清。多写一个临时变量几乎不花钱。

本讲不展开调试步骤，这里只给一次检视示例，说明“置空”在调试器里是什么样子：

```
$ lldb -b ./list_free -o "breakpoint set --name freeList" -o run -o "p *head" -o finish -o "p h1" -o "process kill" -
o quit
Breakpoint 1: where = list_free`freeList + 12 at list_free.c:53:20
(lldb) p *head
(Node *) 0x00005555555592a0
(lldb) p h1
(Node *) NULL
```

进 `freeList` 之前 `*head` 是一个有效地址，函数返回之后 `h1` 已经是 NULL，这一行就是“置空”的现场。

详解 18

NULL、悬空指针与解引用的安全边界

指针的安全边界是本讲最容易出错、也最容易忽略的地方。

三个术语先定义清楚：

- **空指针**：取值 NULL 的指针，它不指向任何有效对象。判空是链表中每条遍历、每次取成员之前的常规动作。
- **悬空指针**（dangling pointer）：曾经指向一块有效内存、而那块内存**已经被释放**的指针。它的值不一定等于 NULL。
- **野指针**（wild pointer）：从未被初始化、值不确定的指针变量。它的值同样不一定等于 NULL。

解引用的安全边界：`*p`、`p->data`、`p->next` 都要求 `p` 指向一块**仍然有效且属于本程序**的内存。这条边界与“是不是 NULL”是两件事：

- NULL 判空只能拦住空指针。
- 悬空指针与野指针的值通常**不是** NULL，判空对它们毫无反应，代码会一路走到“读一块已经被释放或从未属于程序的内存”上去。

所以“每次用之前判一下空”是不够的，真正的边界是**所有权**：谁 `malloc`、谁 `free`，并且 `free` 之后必须让所有还持有这个地址的变量失效（置空），让“这块内存还能不能碰”这件事变成可以从变量取值判断出来的事实。

反例一：先 `free` 再访问。 `examples/list_uaf.c` 释放整条链之后直接读 `head->data`：

```
freeListNoNull(head); /* 反例第一步：释放了全部节点，没有把 head 置空 */
printf("释放后 head 仍然是 %p\n", (void*)head);

/* 反例第二步：直接解引用已经释放的内存 */
```

```
printf("head->data = %d\n", head->data);
```

用 clang 的 AddressSanitizer 编译运行，程序当场报错并终止（节选）：

```
$ clang -Wall -Wextra -g -fsanitize=address list_uaf.c -o list_uaf_asan
$ ./list_uaf_asan
释放前: 0 -> 1 -> 2 -> 3 -> NULL
释放后 head 仍然是 0x559ab38f22a0
=================================================================
==1342843==ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010
READ of size 4 at 0x502000000010 thread T0
    #0 0x561f4e9e3d4f in main list_uaf.c:75:37

0x502000000010 is located 0 bytes inside of 16-byte region
freed by thread T0 here:
    #1 0x561f4e9e3be0 in freeListNoNull list_uaf.c:58:5

previously allocated by thread T0 here:
    #1 0x561f4e9e38a4 in createNode list_uaf.c:19:26
    #3 0x561f4e9e3cd2 in main list_uaf.c:66:16

SUMMARY: AddressSanitizer: heap-use-after-free list_uaf.c:75:37 in main
==1342843==ABORTING
```

这份报告把三件事一次说清：错误类型（heap-use-after-free，读已释放的堆内存）；出错位置（`main` 第 75 行读 4 个字节，正是 `head->data`）；这块内存的来龙去脉（在 `freeListNoNull` 第 58 行被释放，最早由 `createNode` 申请）。

这一类错误之所以危险，是因为不加检测时它经常什么都不报：不加检测的版本程序正常退出、退出码 0，只是读出了一个垃圾值，而且每次运行都不一样（本机连续三次读到 1504393458、1684114980、1558492372）。语言标准对访问已释放内存没有规定任何行为，所以这样的读数既不能当作有效数据，也不能用“上一次运行没问题”当作代码正确的证据。

反例二：忘记置空。 上面那个 `freeListNoNull` 的形参是单指针，只负责释放：

```
1 void freeListNoNull(Node* head) {
2   Node* current = head;
3   Node* temp;
4   while (current != NULL) {
5     temp = current->next;
6     free(current);
7     current = temp;
8   }
9 }
```

函数本身没有错，错在它**没法把调用者的 `head` 置空**：形参是值传递的一份拷贝，改它不影响调用者。这个值看上去毫无异常，用 `printf` 打印、用 `if` 判空都不会报错，唯一的后果藏在“下一步要不要解引用它”这个决定里。

野指针从哪来，四条常见来源：

1. 声明了指针变量却没有初始化，就直接使用它的值；
2. `free` 之后没有置空（反例二），此后它既是悬空指针，也容易被误当成野指针；
3. 返回了局部变量的地址（第 1 小节 `createNode` 必须用 `malloc` 的理由）；
4. 越界写坏了一个指针域（例如数组越界覆盖了相邻的 `next`），此后整条链的走向都由被破坏的值决定。

这四条的共同点：**它们都不会在出错的那一行报错，而是在很远的后面才发作**，所以只能靠纪律与工具防，不能靠读代码防。

四条纪律：指针声明时立即初始化（不确定就先置 NULL）；`free` 之后紧接着把该变量置空；释放函数一律接收二重指针并显式置空调用者的 head；含指针的实现至少用 `clang -fsanitize=address` 跑一遍。

详解 19 Q 8

Julia 侧的链表：抽象型定义与操作

Julia 侧的链表：抽象型定义与操作

上一节的链表长在 C 的语言里：节点是结构体，链接是指针，空值是 `NULL`，节点的创建与释放要自己写。本节换一种语言看同一种结构。Julia 侧的做法是：用类型把空链与非空链分成两种，用引用把节点连起来，把节点的生命周期交给垃圾回收。

换语言不是为了把同一段代码再写一遍，而是为了让链表里与语言无关的三条线显出来：结构（空链与非空链）、次序（`head` 与 `tail`）、以及改写一条链与构造一条新链的分别。本节只用一套记法：抽象类型 `LinkedList{T}`、空链 `Nil{T}`、非空链 `Cons{T}`，再加两个辅助构造函数 `cons` 与 `nil`（这两个名字来自 Lisp，链表的记法传统，下一讲展开）。节内所有输出都来自真实运行：Julia 侧的脚本是 `examples/l03-050-julia-abstract-list.jl`、`examples/l03-050-julia-ref-semantics.jl`、`examples/l03-050-julia-nothing-null.jl`、`examples/l03-050-julia-dispatch-nil.jl`（用 `julia` 运行），类型与操作的完整定义集中在 `examples/l03-050-julia-linkedlist.jl`，由前三个脚本包含；C 侧对照的两段脚本是 `examples/l03-050-null-check.c` 与 `examples/l03-050-null-deref.c`（用 `clang -Wall -Wextra -g` 编译后运行）。

抽象型链表：用两个类型把空链与非空链分开

C 侧的节点结构体同时承担两种情形：装着元素，或者链到此结束。区分靠的是指针域里的一个特殊取值 `NULL`，所以这条链是不是空的，要由数据本身来回答。Julia 侧把这个区分换个位置放：把空链与非空链做成两个具体类型，再用一个抽象类型把它们统起来。

三个类型与两个构造函数的定义如下（完整定义与全部输出见后面的详解块）：

```
abstract type LinkedList{T} end

mutable struct Nil{T} <: LinkedList{T}
end

mutable struct Cons{T} <: LinkedList{T}
  head::T
  tail::LinkedList{T}
end

cons(h, t::LinkedList{T}) where {T} = Cons{T}(h, t)
nil(T) = Nil{T}()
```

这里有五个术语是第一次出现，先给完整定义：

- **抽象类型**：不能直接创建实例的类型，它只用来把一组具体类型归到一个共同的名字之下，并允许拿这个名字写方法。`LinkedList{T}` 就是这样一个名字。
- **类型参数**：花括号里的 `T`，表示链表中元素本身的类型。`LinkedList{Int64}` 与 `LinkedList{Float64}` 是两个不同的类型。

- **具体型**：可以创建实例的类型。本节有两个：`Cons{T}` 表示非空链，`Nil{T}` 表示空链。
- **辅助构造函数**：不直接用 `Cons{T}(...)`、`Nil{T}()` 这样的长名字，而是提供 `cons` 与 `nil` 两个短名字的构造函数。
- **head 与 tail**：非空链的第一个元素与它剩下的子链。`head` 是元素本身（类型为 `T`），`tail` 仍是一条 `LinkedList{T}`，它可能是 `Cons`，也可能是 `Nil`。

把一条链构造出来并观察它：

```
l = list(1, 2, 3)
typeof(l)          # Cons{Int64}
eltype(typeof(l))  # Int64
head(l)            # 1
tail(l)            # list(2, 3)
length(l)          # 3
isempty(list())    # true
```

有两处值得停下来看。

第一处是 `typeof(l) = Cons{Int64}`：元素类型写进了类型里。`list(1, 2, 3)` 得到的是元素为 `Int64` 的链，`tail(list(1))` 打印成 `list()`，它是一个 `Nil{Int64}`，空链也带着元素类型参数。

第二处是空链取首元素的后果：`head(nil(Int))` 抛 `MethodError`，`first(list())` 抛 `ArgumentError`，两者都是带类型与错误信息的异常对象，可以被 `try` 接住。也就是说，空链交给只对非空链定义的方法时，错误当场出现；没有像 `NULL` 那样的特殊返回值，让错误在后面几步之外才暴露。

那么，把空做成一个类型、把非空做成另一个类型，动机在哪里？

1. **判空从每个函数里消失，交给类型分派。** **类型分派**指同一个函数名按实参的类型选择不同的方法体。在 C 侧，取首元素、求长度、打印这些操作都要先问一句 `next` 是不是 `NULL`；在这里，`head` 与 `tail` 写成只接受 `Cons` 的方法，空链根本进不来；`length` 写两个方法，一个给 `Nil` 直接返回 `0`，另一个给 `Cons` 走遍历，两个方法各自独立，没有一句 `if`。
2. **是空的这件事从数据里挪到了类型里。** C 里 `NULL` 是一个指针的特殊取值，空由数据携带；这里空链是一个对象 `Nil{T}()`，空由对象的类型携带。
3. **代价要认。** 用类型换掉判空不是免费的：方法签名与元素类型的显式定义都要多写；操作很少的时候，一句判空可能更省。哪些地方要多写、多写成什么样，页末展开里逐条给出。

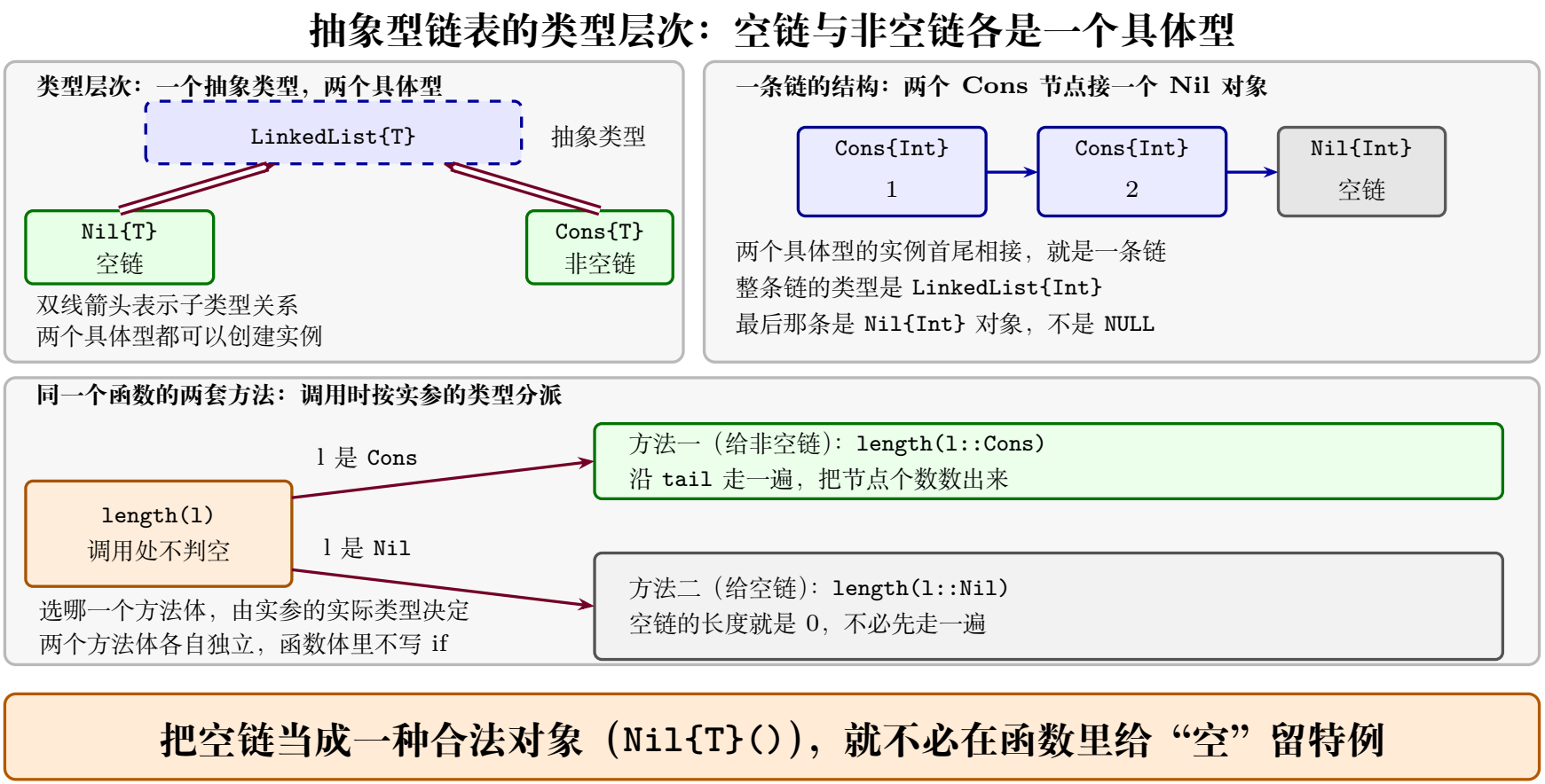


图 11：抽象型链表的类型层次与方法分派

本节配三道练习，都放在折叠起来的详解块里，请先自己写一遍再用脚本核对：练习一（构造与分派）在下面的第一个详解块，练习二（引用语义与共享）在第二部分末尾，练习三（用 cons 把操作写出来）在第三部分末尾。

详解 20 **详解 21** **详解 22**

三处容易误解的地方

这套抽象型链表的接口很短，误解集中在三处：变量与对象的关系、改写一条链的代价、以及空值到底是什么。

一、引用语义：变量绑定到对象，赋值不复制对象。

`l2 = l` 之后，两个名字指向的是同一个对象，不会把整条链复制一份；通过其中一个名字改到节点，另一个名字看到的是改后的结果。判据是 `===`：它为真就说明两个名字是同一个对象。

```
a = 1
b = a
b += 1                # a = 1, b = 2，改 b 不影响 a

v1 = [1, 2, 3]
v2 = v1
push!(v2, 4)          # v1 跟着变成 [1, 2, 3, 4]，v1 === v2 为 true

l = list(1, 2, 3)
m = l                 # m === l 为 true
m.tail.tail = nil(Int)   # 通过 m 改节点
# l 与 m 都打印成 list(1, 2)
```

容易被误解的地方是把别的语言的经验搬过来：C 里 `Node b = *a;` 会把整个结构体的内容复制一份，而 Julia 里 `b = a` 永远只复制绑定，不复制被绑定的对象。区别不在赋值这个动作，在被赋值的东西：`Int` 这类不可变的值，赋值复制的是值本身；数组、链表节点这类可变对象，赋值复制的是指向它的名字。

二、不可变接口下的修改：实际是构造一条新链，并与原链共享后缀。

这套接口只提供 cons 与 nil 两个构造函数、head 与 tail 两个取值函数，接口里没有原地往表头插一个元素这样的操作。在表头加一个元素的写法是 cons(0, l)：新建一个节点，新节点的 tail 直接就是原来的整条链。

```
l  = list(1, 2, 3)
l2 = cons(0, l)          # l2 = list(0, 1, 2, 3)
# l2 =/= l 为 true
# l2.tail === l 为 true
# l2.tail.tail === l.tail 为 true

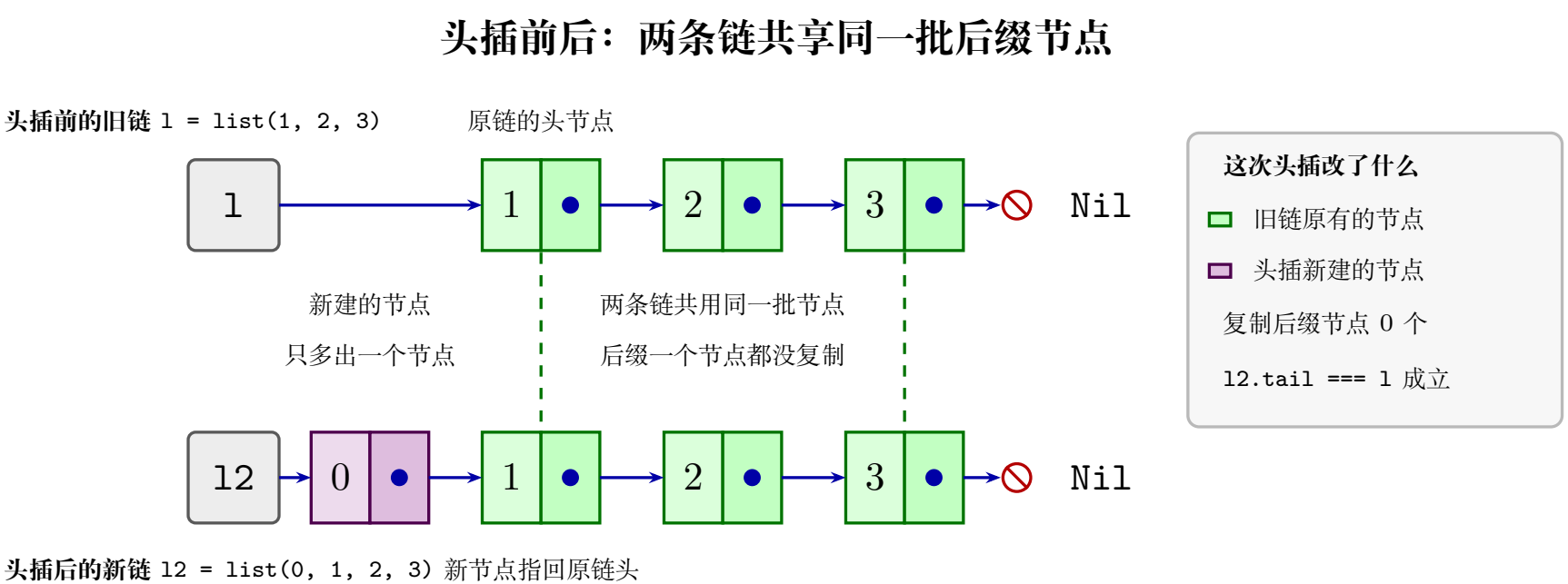
c = copy(l)
# copy(l) == l 为 true
# copy(l) === l 为 false
# copy(l).tail === l.tail 为 false
```

结构共享指多个版本共用同一批节点。cons(0, l) 只多出一个节点，后缀一个节点都没复制；copy 的实现是 reverse(reverse(l))，两次反转各建一批新节点，所以副本与原链没有任何共享。整条链规模为 10000 个节点时实测：cons(0, l) 分配 32 字节（一个 Cons{Int64} 实例本身 24 字节，差别来自分配记账），与链长无关；copy(l) 分配 640016 字节，与链长成正比。

与 C 侧对照：在表头插入同样是 O(1)，但 C 侧改的是两个指针（新节点的 next 指向原 head，head 指向新节点），改完只有一条链；这里换来的是一对共享后缀的链，旧版本一个节点都没被改坏。

有一处必须说清，否则会与另一种写法打架：那种写法把 Nil 与 Cons 两个具体型都声明为 mutable struct，也就是字段本身可以被重新赋值，原地插入排序正是靠直接改节点里的 tail 字段。所以这里说的不可变指的是**接口**：接口不提供原地插

入与删除，改写只能靠 cons 构造新链；字段能不能改，是这套定义留给后文实验的余地。本节主线只用 cons 与 nil 这套接口。



在表头插入不改已有节点，旧链仍然完整可用，这就是不可变结构共享的意义。

图 12: 头插前后两条链共享同一批后缀节点

三、nothing 与 C 里 NULL 的差别。

`nothing` 是 `Nothing` 类型的唯一实例，是一个普通的值：可以作为字段的值、函数的返回值、容器的元素，可以判等，也可以为它定义方法。C 的 `NULL` 是指针类型的一个特殊取值，只有指针类型能拿它作比较，对 `NULL` 解引用是未定义行为。

```
typeof(nothing)          # Nothing
nothing isa Nothing      # true
nothing === nothing      # true
isnothing(nothing)       # true
Vector{Union{Nothing,Int}}([1, nothing])   # Union{Nothing, Int64}[1, nothing]
```

用 `Union{Nothing, 节点型}` 表示链尾的写法（本节只作对照，主线用的是 `Nil{T}`）：

```
mutable struct ListNode
  val::Int
  next::Union{ListNode, Nothing}
end

n3 = ListNode(3, nothing)
n2 = ListNode(2, n3)
n1 = ListNode(1, n2)

n1.next.next.next            # nothing
n1.next.next.next === nothing   # true
n2.next.val                  # 3
```

误用的后果，实测是一个可以接住的异常：`n3.next.val` 抛出 `FieldError(Nothing, :val)`，`nothing + 1` 抛出 `MethodError`。对照组是 C 侧：把 `NULL` 拿去解引用（`examples/l03-050-null-deref.c`），段错误退出，实测退出状态 139，没有任何异常可以

捕获。区别就在这里：这里的错误带类型与信息，可以说清错在哪里；C 侧的未定义行为可能当场崩溃，也可能静默给出一个错的结果。

还要分清一件事：`nothing` 与空链不是同一个概念。主线里空链是对象 `Nil{T}()`，它仍然是一条链，实测 `length(nil(Int)) = 0`、`collect(nil(Int)) = Int64[]`、`for` 遍历它一次也不走；`nothing` 不是链，把 `nothing` 交给链表的操作会抛错。`Union{Nothing, 节点型}` 那套写法表达的是这个字段可能没有值，与这是一条空链不是同一个说法。

详解 23 **详解 24** **详解 25**

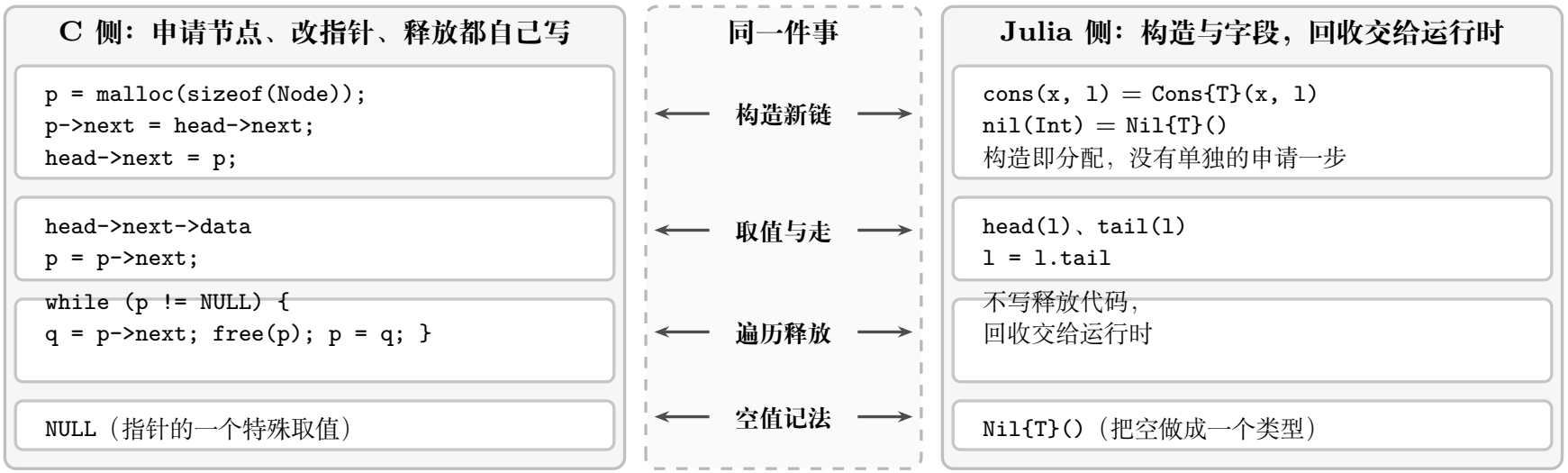
Julia 侧的链表操作

在 Julia 侧，可用的链表操作不是在某个现成的链表库上查表，而是把这些操作以方法的形式定义在抽象类型 `LinkedList` 上：定义一次，两个具体型按类型分派各走自己的实现。这样做的好处在写题目时最明显，长度、判空、取首尾、转成向量、映射、过滤、反转、复制、判相等这些常用操作都有现成的方法，直接写一行。

与 C 侧手写指针的对应关系，两句话就够：同一个操作，在 C 里是一段自己改指针的代码，在 Julia 里是给抽象类型加一个方法；分派还替写出了一部分特例，例如 `length` 对空链直接返回 0，不必在遍历之前先判空。

一句提醒：Julia 标准库并没有链表类型，本节列的这些方法就是 `examples/l03-050-julia-linkedlist.jl` 里定义的那些。

同一批操作：C 侧手写指针，Julia 侧方法名



两边做的事相同，
差别在谁管内存、空值怎么写

学会一边就能推另一边，但别把两边的空值记法混用

图 13: C 侧手写指针与 Julia 侧方法名的对应

详解 26 详解 27

基础练习与应用：ASEP 的链表实现（不使用数组）

基础练习与应用：ASEP 的链表实现（不使用数组）

上一节把 Julia 侧的抽象型链表讲完，链表的写法与代价都摆到了台面上，但用到的规模都很小，改动只落在几个节点上。本节把链表放进一个要连续跑几千步的模拟里：一维格点链上的不对称简单排斥过程（ASEP）。**本节末尾就是正式作业的布置：ASEP 的链表实现是本讲的课后作业题二（本周核心练习），当堂的改写练习与完整实现都要按验收要求交。**

选这道题有三条理由。第一，它的数据结构天然是一条链：一维格点逐个相接，每个粒子只与相邻格点发生关系。第二，它每一步都要读邻居、判空、改状态，用到的全是本讲前面几节的基本操作，没有一处能靠下标绕过去。第三，规模与步数的要求足够大（格点数 50 以上、步数 5000 步以上），写法上的失误会在几千步里反复出现；而它的正确性可以用手算与守恒量核对，不需要统计物理的背景。

口径先行：本节要求格点与粒子都用链表表示，**不使用数组**。本节所有数值与输出都来自实跑：脚本 `examples/asep-linked.jl`（用 `julia` 运行，本机 julia 1.12.6，整份脚本一次运行约 0.9 秒），轨迹数据在 `examples/asep-trajectory.txt`。

ASEP 是什么：模型定义与练习要求

定义：ASEP（Asymmetric Simple Exclusion Process，不对称简单排斥过程）是一条一维格点链加若干粒子，粒子按不同的概率向两个方向尝试跃迁，只有目标格点为空时移动才成功。四个术语在这里第一次出现，先给完整定义。

- **格点**（site）：格点链上的一个位置，最多容纳一个粒子。格点链的长度记作 L，格点按顺序相接。
- **粒子**（particle）：占据在某个格点上的对象，本节用一个编号标识它。粒子数记作 N。
- **排斥原则**（exclusion）：每个格点最多只能有一个粒子，两个粒子不可能同时待在同一个格点上。
- **周期性边界**（periodic boundary）：链的两端相接，粒子从一端走出就从另一端出现，因此链上每个格点都有两个邻居格点，没有端点特例。

模型的三条核心特征：**排斥**指每个位置最多一个粒子，这条把粒子之间唯一的相互作用限制为阻断；**随机**指粒子是否移动由概率决定；**不对称**指向前与向后的尝试概率不相等，记作 $P_{\text{前进}}$ 与 $P_{\text{后退}}$，两者不等就意味着系统有一个偏向，整体会出现净流动。密度是一个派生量：$\rho = N/L$，表示被占据的格点比例。

它模拟什么。 三处真实场景可以直接对应到这三个参数上。交通流把每个格点看成一段路、把粒子看成一辆车、把空格点看成空出的路面：ρ 就是车流密度，向前的偏向对应所有车朝同一个方向开，高密度下空格点稀少，粒子频繁互相阻断，模型就会给出走走停停的拥塞。生物分子马达把格点链看成一条轨道、把粒子看成在轨道上步进的马达：马达沿轨道有方向偏好地前进，且不能与前一个马达重叠。非平衡系统中的传输过程把格点链看成一段受驱动的通道：系统不断被驱动、不断耗散，不会像无驱动的随机行走那样走向静止分布。ASEP 把驱动、随机、排斥三条约束装在最小的结构里却仍可解析处理，因此是非平衡统计物理的标准范例（模型出处：Mallick, *The Exclusion Process: A paradigm for non-equilibrium behaviour*, Physica A 418, 2015）。**本节只把它当数据结构练习题的来源，不展开统计物理的推导与解析结果**。这一步的边界划在这里，是因为本节要练的是链表，不是物理。

为什么放进链表课。 在编程意义上它是一个元胞自动机（Cellular Automaton）：格点上的离散状态按固定规则逐步更新。元胞自动机的通常写法会用一个下标数组存状态，因为“第 i 个格点的邻居”可以按 $i-1$ 与 $i+1$ 算出来。本节的要求是把这件事整条搬到一个链表表示上：一维格点链就是一条链表，相邻关系就是链上的 `next` 关系，周期边界就是首尾相接。判断一个格点是否为空、把粒子从一个格点挪到相邻格点，落到链表上分别是一次读取与一次标记改写；而“找到某个格点的另一个邻居”这件事，在链表上只能靠走链，正是本讲反复讲的顺序访问。题目因此把一周的基本操作全部用上，并且把它压在一个要跑几千步的程序里。

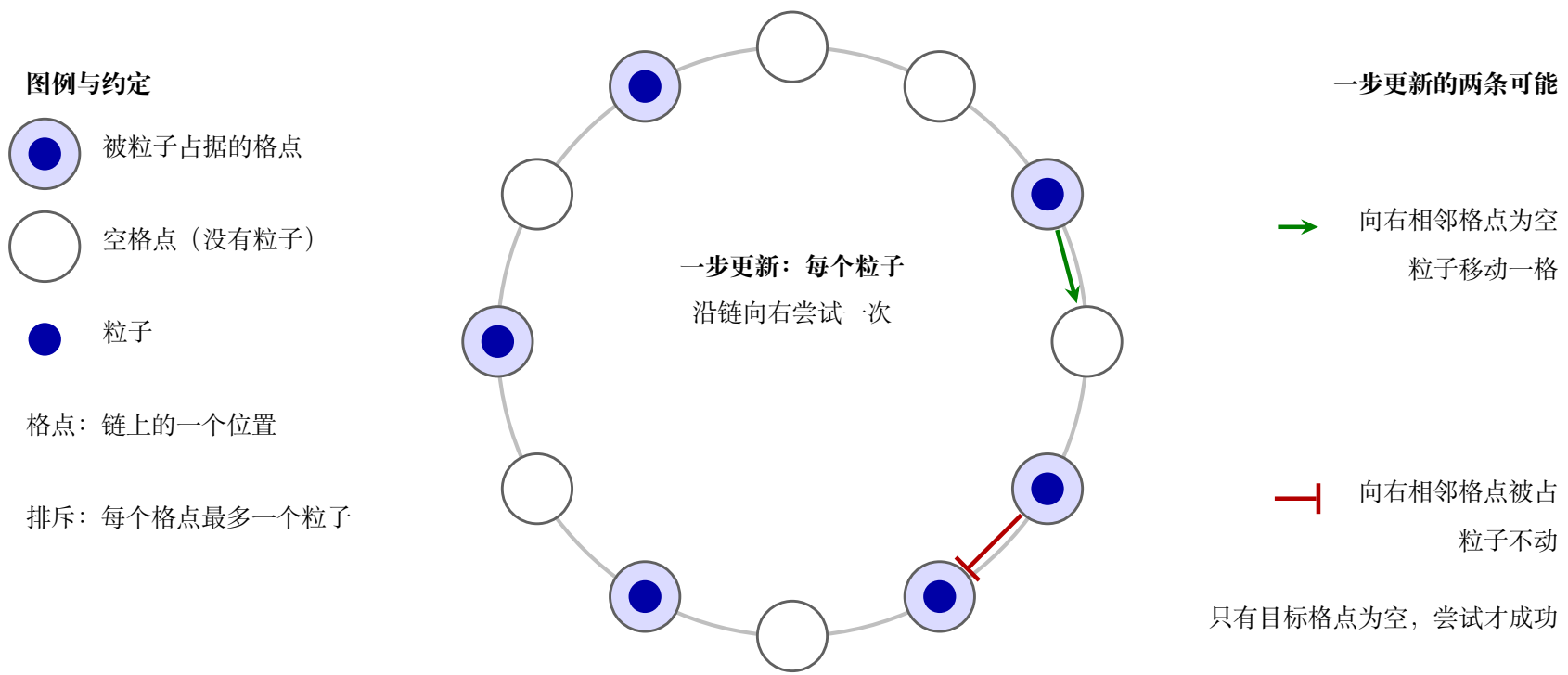


图 14：ASEP 的格点环与一步更新的两条可能

Q 9 AI 3 详解 28

表示上的三个判断（辅导分析，实现前先想清这三件事）

模型定下来之后，剩下的全部工作是表示与实现。表示上有三个判断必须先做，做错了后面改不动。

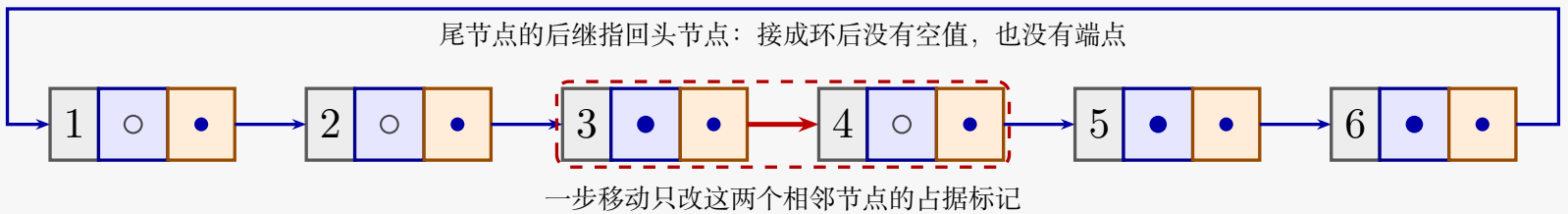
判断一：格点链，还是一串粒子记录。 两种候选都像链表。**格点链**是 L 个节点，每个节点代表一个格点，节点内保存占据标记（这个格点上有没有粒子、是哪一个）与后继引用；**粒子链**是只把被占据的格点做成节点，每个节点记一个位置，空格点不出现在链上。判据是判空的代价：在格点链上，判空是读一个节点的标记，代价 $O(1)$；在粒子链上，判空的含义变成“某个位置在不在链上”，而链表上没有随机访问，只能沿链找，代价 $O(L)$。结论是取格点链：节点数恒为 L，与粒子数无关，空格点也各占一个节点，代价是每个节点多保存一个标记。粒子链在 ρ 很小的稀疏场合能省内存，但把每一次判空都变成一次走链，代价方向正好相反。

判断二：周期边界怎么表达。 两种候选。**首尾相接**是把尾节点的后继指回头节点，环在建立链表的时候一次接完，此后每个格点的两个邻居都可以从链上直接拿到，边界不出现特例。**边界处特判加取模的位置下标**是每次移动都判断是否跨界，跨界时把位置折回另一端；这条路在数组上很自然，在链表上不省任何东西，因为“取第 k 个格点”本身要沿链走 k 步，取模只是把回绕写成一个算式，走的步数一步没省。结论是取首尾相接。代价必须一起讲清：接成环之后链上不再有空值，走链不能以“遇到空值”作为终止条件，必须以格点数计数终止，计数写错就是死循环。常见的链表错误里有一类是“不小心造成环导致遍历死循环”；这里的环是有意造的，判断它是哪一种只能靠“终止条件是什么”。

判断三：向后尝试要走几步。 向后尝试的目标格点是当前格点的前驱。两条路：走链时同时持有前驱与当前（两个变量同步前进），一个循环里就把两个邻居都拿到了，向后尝试另付 0 次访问；反过来，每次向后尝试都从 head 重新走一遍去找前驱，单次向后尝试的代价就从 1 次访问涨到平均 $L/2$ 次。实测（$L = 50$，$N = 40$，同一种子，只算决策那一遍走链访问的格点数）双指针版 50 次，反例版 348 次，而这一步里只有 11 次向后尝试；粒子更多的模型按 $N \times L/2$ 增长，5000 步的规模下会直接拖垮运行时间。这条判断与本讲前面删除操作的结论是同一件事：单链表上前驱只能靠走链拿到。

格点环的链表表示：6 个格点接成一条首尾相接的单链（状态取自 L = 6、N = 3 演示的步 1）

占据标记：● 有粒子，○ 空格点；右侧一格是后继引用



格点 3 由占变空、格点 4 由空变占；向后一步同理，改动的仍是相邻的一对

轨迹文件的体例（examples/asep-trajectory.txt 的前三行）

步数	1	2	3	4	5	6	7	8	9	10	11	12	13	14	...
0	1	2	3	4	6	7	8	9	11	12	13	14	16	17	...
1	1	2	3	5	6	7	8	10	11	12	13	14	16	17	...
2	1	2	4	5	6	7	8	10	11	12	13	14	16	17	...

第一列是步数，其后是各粒子的格点编号；
文件里共 40 个粒子，图中只截到第 14 个；
把某一列看下去就是单个粒子的 x(t)；
图中数字按列摆开，原文件里只隔空格。

每一行与上一行的差别就是这一步的成功跳跃：图中第 4 列 4 变 5、第 8 列 9 变 10。
画图时横轴取步数、纵轴取格点编号，把 40 条线画在同一张图上。

图 15: ASEP 的链表表示与轨迹文件体例

一步的口径也要先写清。 实现要求说“每一步对所有粒子各尝试一次跳跃”，这句话必须补一条口径才没有歧义：目标格点在步首为空才算成功；同一个大步里两个粒子同时选中同一个空格点时，两个都不动。这条口径的理由是可比较性：同名的模型在不同更新口径下给出不同的流动，口径不写清，两次运行的统计量没有对照的意义。口径本身可以选平行更新

（判定基于步首的状态）或顺序更新（判定基于当时最新的状态），两条都合法，但**作业必须写明用的是哪一条，且整份实现只用一个口径**。本节参考实现用平行更新加“同格点冲突则都不动”这条规则。

详解 29

实现要求六条（作业题二的正文要求）

1. 支持周期性边界条件，粒子到达一端时从另一端出现。
2. 系统长度 $L \geq 50$，粒子密度 $\rho = N/L$ 满足 $0.5 \leq \rho \leq 0.9$。
3. 跳跃概率满足 $P_{\text{前进}} \neq P_{\text{后退}}$，且两者取值都在 $[0.2, 0.8]$ 之内。
4. 每次模拟运行的步数不少于 5000 步，每一步定义为对所有粒子各尝试一次跳跃。
5. 把所有粒子的轨迹 $x(t)$ 画在一张图里，横轴是步数，纵轴是格点编号。
6. 语言限定 C 或 Julia；**格点与粒子都用链表表示，不使用数组**；把链表结构与算法封装成模块便于复用与测试；代码要有注释。

验证方式五条（作业的验收依据）

判空、改标记这类操作在链表上没有现成的正确性检查，唯一的办法是把模拟的可观测量拿出来对账。作业提交的说明文档里必须逐条给出期望值、实测值与结论。

1. **小规模手算对照。** 取 $L = 2$、$N = 1$、$P_{\text{前进}} + P_{\text{后退}} = 1$：链上只有一个粒子，另一个格点必定为空，因此粒子每一步都必定移动；两个方向的目标都是同一个格点，所以结论与随机数无关，粒子在第 1、3、5 步落在 2 号格点、在第 2、4、6 步落回 1 号格点。实测 10 步完全符合，位串按 `x.` 与 `.x` 交替。
2. **守恒检查。** 格点数与粒子数在整个模拟里都不变；每个格点的占据数始终不超过 1。实测 $L = 50$、$N = 40$、5000 步：粒子数始终是 40，占据数上限始终是 1。
3. **漂移方向检查。** 单个粒子时没有阻断，净位移就是“向前次数减向后次数”，期望值等于 $(P_{\text{前进}} - P_{\text{后退}}) \times$ **步数**，一步位移的方差等于 $P_{\text{前进}} + P_{\text{后退}} - (P_{\text{前进}} - P_{\text{后退}})^2$。实测 $L = 50$、$P_{\text{前进}} = 0.7$、$P_{\text{后退}} = 0.3$、5000 步：向前 3494 次、向后 1506 次、净位移 1988，期望 2000，5000 步的标准差 64.8，偏差 −12 也就是 −0.185 个标准差。方向为正，与不对称的偏向一致。
4. **固定种子复现。** 同一个随机数种子下两次运行的摘要必须逐字相同。实测两次都是 2000 步累计跳跃 11241 次，末态位串也逐字相同。**随机数种子**是随机数生成器的初始状态，给定它之后整条随机序列就定下来，因此“可复现”指的是同一份代码、同一个种子、同一台机器上的同一串结果。
5. **账目闭合。** 每次尝试只有三种归属：因目标格点被占而失败、因两个粒子抢同一个空格点而失败、成功跳跃。三者的和必须等于尝试总数。实测 5000 步：尝试 $5000 \times 40 = 200000$ 次，其中目标被占失败 154612 次、抢同一空格点失败 17290 次（8645 处冲突，每处两个粒子）、成功跳跃 28098 次，三数之和正好是 200000。这条检查最有用，因为它能抓住一整类错误：粒子数忽然少了（有人把标记盖掉了）、成功数多于尝试数（有人一条粒子多动了一次）、冲突数长期为零（双向模型里不可能为零，说明后退这一路根本没接上）。

关于对照方式还有一处要交代。另一种做法是在同一随机种子下与数组版对照统计量。本作业硬要求不使用数组，因此主体核对改为与手算期望值、守恒量与固定种子复现对照，本节也不提供数组版实现。若读者手上另有按数组版写过的实现，可以拿它当参照物做一次统计量对照，但提交版本只用链表。

详解 30 详解 31

参考实现要点与实跑（辅导材料，实现要自己写）

参考实现 `examples/asep-linked.jl` 的骨架思路：格点链用一条首尾相接的节点链，节点保存占据标记（0 表示空格点，否则是粒子编号）与本步的方向、冲突登记字段；一大步分三遍走链（清登记、抽方向并登记申请、按申请数裁决移动），每遍 L 个格点；抽到方向之后必须先看目标格点是否为空，不能先登记再筛。完整代码在 `examples/` 下，**本节不抄进正文**：把它逐行抄成自己的实现等于没做题。

两个值得看的实跑现象（输出全文可在 `julia asep-linked.jl` 复现）：

1. **漏判空的后果能被守恒检查当场抓住。** 反例脚本 `examples/asep-bug-target-check.jl` 与参考实现只差“目标为空才登记”这一句检查，实跑粒子数从 40 一路掉到 1：两个粒子先后登记同一个目标格点，后一个的编号覆盖了前一个的标记，粒子凭空消失。格点数不变、位串照常打印，只有守恒检查能抓住它。
2. **成功跳跃只占尝试总数的约 14‰。** $\rho = 0.8$ 的 5000 步里 200000 次尝试，成功 28098 次（向前 23069、向后 5029），其余全部被排斥挡住，这就是高密度交通流寸步难行的来源；向前成功与向后成功之比 4.6，比抽签比 0.7 : 0.3 悬殊得多，因为一个移动要成功，还得它的对手不往那边挤。

详解 32

作业布置：ASEP 链表实现（本讲作业题二，本周核心练习）

把上面的实现要求与验证方式正式布置为作业，随本讲其它三题一并提交：

- **题目**：用链表实现一维 ASEP（不使用数组），六条实现要求与五条验证逐条满足，提交完整源码、轨迹数据文件、绘图脚本，以及按五条验证逐条写的核对报告（期望值、实测值、结论）。
- **交付物**：`asep.jl` 或 `asep.c`（含链表结构与封装）、`trajectory.txt`、绘图脚本、`README.md`（结构说明、运行方法、所用更新口径、种子值）。
- **验收**：五条验证的实测数字与你的说明写进 README；`clang -Wall -Wextra -g` 零告警（C 侧）或 `julia` 直接跑通（Julia 侧）；固定种子两次运行逐字可复现；代码里不出现数组作为格点或粒子的存储结构。
- **提交与截止**：按本讲末尾“课下阅读与练习布置”一节的课程 Git 流程，周日晚上截止，评分四档以课程考核文件为准。
- **提醒**：先做练习二第一问（纸面改写四答）再动手写代码；口径（平行/顺序更新、冲突规则）写进注释与 README，否则你的统计量与任何人（包括下一次的你）都不可对照。

常见错误与课堂练习（第一周）

常见错误与课堂练习（第一周）

前面几节把链表的结构、基本操作、C 与 Julia 两边的落地以及 ASEP 的链表表示依次立起来了，这一节做两件事，次序不能颠倒：先当堂做题，做完了再回看错误清单与经验。理由是这些错误只有当自己在指针上踩过一次，读起来才不是一句口号；把清单放在练习之前，读完就忘。全节的顺序是：先给一道示范把题目走完整，再成组给出第一周的课堂练习三题，最后收在常见错误与重要经验上。

三题只用第一周讲过的东西：指针、动态内存与结构体（第 1 周），节点定义、插入与删除、由数组建链、遍历打印、释放（本周），以及本周新增的 ASEP 链表表示（上一节）。凡是需要把多个基本动作组合起来、或者要动到整链重排的题目都不在本周之列。本节引用的代码在 `examples/l03-070-practice.c`、`examples/l03-070-practice.jl` 与 `examples/l03-070-asep-rewrite.jl`，引用的输出与计数全部来自实跑（clang 19.1.7 与 julia 1.12.6）。

课堂练习（第一周）：基本操作三题与 ASEP 改写

先看示范，它是练习二的完整过程：把数组 {5, 3, 1, 4, 2} 转成单链表并打印出来。做法是先造哑节点，再用一个尾节点指针记住当前的末尾，之后每接一个元素只改一条指针。

```
/* 由数组一趟建链：先造哑节点，记住尾节点，之后每次只接一步，不重复走链 */
Node* fromArray(const int* a, int n) {
  Node* head = initList();
  Node* tail = head;
  for (int i = 0; i < n; i++) {
    tail->next = createNode(a[i]);
    tail = tail->next;
  }
  return head;
}

void printList(const Node* head) {
  const Node* p = head->next; /* 哑节点不打印 */
  while (p != NULL) {
    printf("%d -> ", p->data);
    p = p->next;
  }
  printf("NULL\n");
}
```

实跑（`clang -Wall -Wextra -g l03-070-practice.c -o l03-070-practice` 之后运行）：

```
数组 {5, 3, 1, 4, 2} 转成链表：5 → 3 → 1 → 4 → 2 → NULL
  一趟建链走链访问的节点数：0
  链表中数据节点的个数：5
反例版（每次插入都从 head 重新走到尾节点）打印结果：5 → 3 → 1 → 4 → 2 → NULL
  反例版走链访问的节点数：10
两版打印结果相同：差别不在结果，在走链访问的节点数
```

走链计数指每沿 next 前进一次就记一次数，它把 $O(n)$ 与 $O(1)$ 落成能数出来的数。这段输出最要紧的是最后一行：反例版的打印结果与正确版一字不差，走链访问的节点数却从 0 次变成 10 次（5 个元素按 $0+1+2+3+4$ 累加）。**结果对不出来的错，才是链表代码里最值得防的错**，因为编译器与运行结果都不会报它。

Julia 侧同一段逻辑（`examples/l03-070-practice.jl`）打印成 `5 → 3 → 1 → 4 → 2 → nothing`，与 C 侧的差别只在空值记法：C 是 `NULL`，Julia 是 `nothing`。

练习一那几件事的现场核对结果如下，走链计数只数沿 next 访问到的节点：

操作（链表为 5 → 3 → 1 → 4 → 2）	改动的指针	访问记数
头插 9	2 条：新节点的 next、哑节点的 next	0
按值查找 3	不改	3
在值为 3 的节点后插入 8	2 条：新节点的 next、值为 3 的节点的 next	0
尾插 7（当时链上有 7 个数据节点）	2 条：尾节点的 next、新节点的 next 置空	7
删除值 8（删中间）	2 条：前驱的 next、free	4
删除值 9（删第一个数据节点）	2 条：哑节点的 next、free	1

操作（链表为 5 → 3 → 1 → 4 → 2）	改动的指针	访问记数
删除值 7（删尾节点）	2 条：尾节点的前一个节点的 next、free	6

访问记数这一列里，插入与查找数的是沿 next 访问到的节点，删除数的是检查过的前驱候选（每检查一个候选记一次）。删第一个数据节点时，被检查的候选就是哑节点后面的那一个，一次就命中，所以是 1；这与基本操作一节里“删头一次都没走链”并不矛盾：走链前进的步数是 0，被检查的候选仍是 1 个。

表里有两行值得停一下。删第一个数据节点只走了 1 个前驱候选，删尾节点走了 6 个：**链表删头便宜、删尾贵**，与插入那一边正好对称，原因都在“要找的位置离 head 有多远”。而且这两件事在带头节点的写法里走的是同一段代码，删头不需要任何特判，这正是哨兵节点买到的东西。

验收判据是当堂核对的四条标准，三题都按它自查：能编译运行（C 侧在 `clang -Wall -Wextra -g` 下到零告警）；输出与手算一致；空链表、只有一个数据节点的链表、删第一个数据节点、删尾节点四种边界有处理；说清每处改动了几条指针、走链访问了几个节点。边界这一项的实跑输出：

```
空链表（只有哑节点）打印：NULL
空链表的长度：0
空链表上删除值 1：0（什么也没删）
空链表上尾插 42：42 → NULL
只有一个数据节点的链表：1 → NULL
删掉它：1，链表回到：NULL
  删完之后链表的长度：0，尾节点的 next 是 NULL：是
两个数据节点的链表：5 → 3 → NULL
先删尾节点 3：1，链表：5 → NULL
  尾节点的 next 是 NULL：是
```

```
再删第一个数据节点 5：1，链表：NULL
```

这里最容易漏的是删尾：删掉尾节点之后，新的尾节点的指针域必须仍然是空值，脚本里的 `tailNextIsNull` 就是在核对它（本机运行全部为“是”）。忘掉这一条，之后的遍历会读出链表之外的地址。

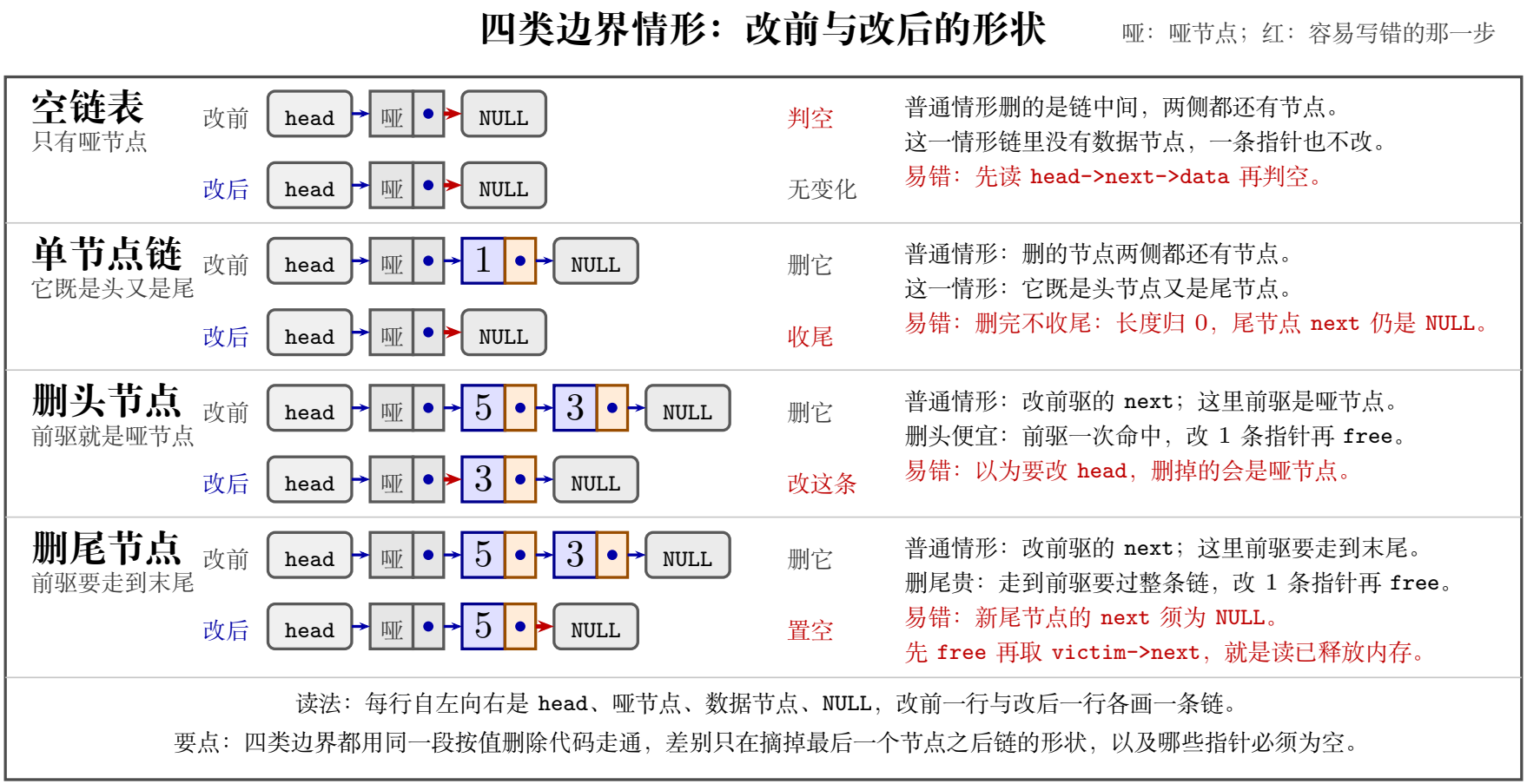


图 16: 链表代码的四类边界情形

Q 10 AI 4 详解 33 详解 34 详解 35

常见错误与重要经验

先做练习再看这一节的顺序不能反：下面五条要逐条指回前面各节真实跑出来的现象，而不是凭空列一张清单。

五条常见错误。 每条都给出它长什么样，以及在本讲前面的哪一次实跑里出现过。

常见错误	它长什么样	前面哪一次实跑里出现过
忘记更新头指针或尾指针	插入或删除之后 head 还指着旧节点，或者尾节点该为空的地方不为空，链表结构已经不完整	基本操作一节：删第一个数据节点时改的是 head 本身；C 实现一节：释放之后不把调用者的 head 置空，就留下悬空指针
删除节点时忘记释放内存	C 侧漏掉 `free`，进程占用的内存只增不减	C 实现一节：`list_basic.c` 用地址检查器跑出 80 字节泄漏，分布在 5 次分配里
空指针访问	对空值解引用：C 侧是未定义行为，本机实测段错误；Julia 侧抛异常	Julia 一节：`l03-050-null-deref.c` 解引用 NULL 之后段错误退出，实测退出状态 139
意外成环	两条指针的赋值次序颠倒，新节点的 next 指向自己，遍历再也不结束	基本操作一节：`bad_order.c` 打印出 25 一直重复到第 10 个节点仍未走到空值，节点 3 与 4 还在内存里但从 head 走不到

常见错误	它长什么样	前面哪一次实跑里出现过
边界条件处理不当	空链表、只有一个节点的链表、删第一个数据节点、删尾节点这四种情形没有分支或分支写错	本页：删尾之后新尾节点的指针域必须仍为空值，脚本里有一句专门核对它

内存这一条另附一句 C 侧的所有权约定：**谁申请谁释放**，`malloc` 与 `free` 成对出现，一次申请对应一次释放；释放之后把调用者的 head 置空。本课不引入智能指针一类的额外机制，理由是本讲要让“内存看得见”这件事本身成为教学内容。

三条重要经验。

1. **先明确链表的结构与节点定义。** 动手写之前把不变量定死：链表是否带头节点、哑节点算不算数据节点、函数要不要接收返回值、空值在两种语言里怎么记。定下来之后所有函数按同一条不变量写，否则会出现“有的函数把第一个节点当数据、有的当哑节点”这种最难查的错误。
2. **用哨兵节点消除头部特判。** 删第一个数据节点与其他位置在程序结构上变成同一件事，特判被消灭而不是被处理。本页第一题的实测正是这件事：两处走的是同一段代码，差别只在位置离 head 有多远（检查过的前驱候选分别是 1 个与 6 个）。
3. **测边界。** 边界的清单是固定的四条：空链表、只有一个节点的链表、插入或删除头节点、插入或删除尾节点。写完一个函数就按这四条各走一遍，不要等整套程序写完再测。

Q 11 AI 5 详解 36

附录（放在正文之后）：分治、递归与循环的思维专题

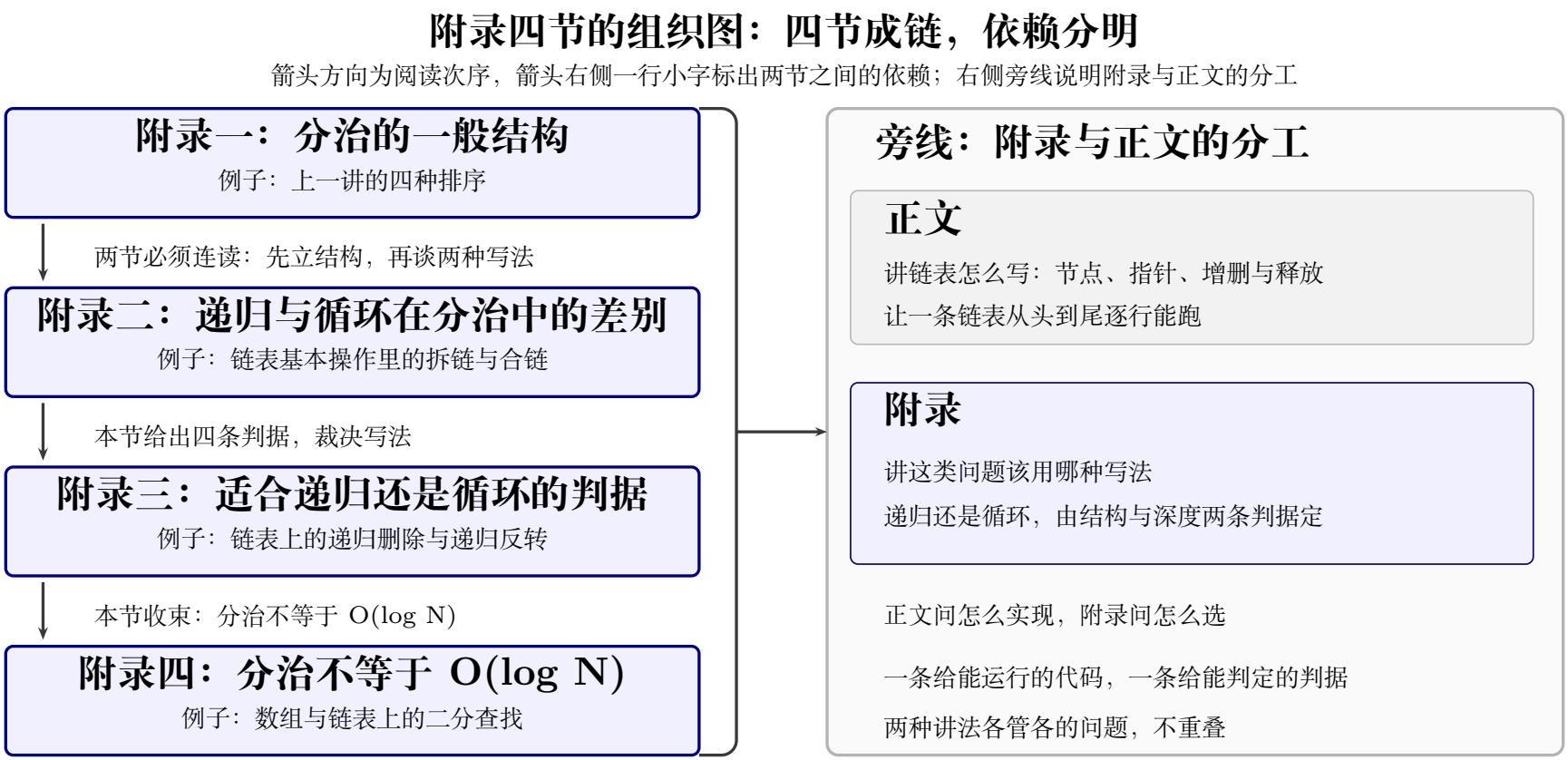
附录（放在正文之后）：分治、递归与循环的思维专题

正文讲链表的写法，附录把正文里反复用到的三条思维收成一般形式、判据与反例。

术语：

- **分治**（divide and conquer，分而治之）指把规模为 n 的原问题划分成若干规模更小的同类子问题，各自求解，再把子问题的解合并回原问题的解的一套做法。
- **递归**（recursion）指函数直接或间接调用自身的写法，它把状态保存在调用栈上。
- **循环**（loop）也叫迭代（iteration），指用循环变量与控制条件重复执行一段代码的写法，它把状态保存在变量里。
- **基线**（base case）指分解到此为止、可以直接求解的最小规模。
- **调用栈**（call stack）是运行时为每个尚未返回的函数调用保存局部变量与返回位置的一块内存区域，其中一层所占的空间称为**栈帧**（stack frame）。
- **弃**（discard）指分完之后依据某种判断把一部分子问题整块排除在后续计算之外。

本附录与正文口径一致：C 用 clang 19.1.7 编译运行，Julia 用 julia 1.12.6 运行；涉及计时与栈大小的数字取自本机，栈上限是 `ulimit -s` 给出的 8192 KiB；量出数字的程序都在 `l03-linked-list/examples/` 下。



要点　附录不是补充材料，是把正文里反复用到的三种思维收成一般形式

图 17: 附录四节的组织与依赖

Q 12

附录一：分治的一般结构

归并排序可以写成一句话：把数组从中间切成两半，各自排好，再把两份有序结果并成一份；这句话里没有循环，也没有递归，它描述的是算法本身的结构，不是某一种实现。

同一个算法的两种写法在同一份输入上实跑，随规模变化的合并次数与元素比较次数如下（`examples/appendix-dc-rec-vs-loop.jl` 与 `examples/appendix-dc-rec-vs-loop.c`，两种语言的计数逐个相同）：

n	合并次数	元素比较次数	递归版的最大递归深度
16	15	46	5
64	63	308	7
256	255	1720	9
1024	1023	8899	11
4096	4095	43490	13

合并次数恰好是 $n - 1$（每一步合并把两份并成一份，n 个单元素子问题并到只剩一份需要 $n - 1$ 次）；元素比较次数与输入的具体排列有关。

从这张表里抽出分治的全部结构，再加前提与终止要求，就是判断一个任务能不能用分治的判据：

1. **分**（divide）：把原问题划成若干规模更小的同类子问题（这里是长度为 n 的区间切成两段）。
2. **基线**（base case）：分解必须终止的位置，规模小到可以直接求解（这里是区间只剩一个元素，天然有序）。
3. **治**（combine）：从基线出发，自底向上把子问题的解合并回原问题的解（这里是两份有序区间并成一份）。
4. **前提一，可分解且同型**：子问题与原问题是同一个问题的更小版本，能按同一套规则递归地划分下去。
5. **前提二，子问题相互独立**：子问题之间不重叠、不互相依赖，可以各自求解。

6. **前提三，可合并**：原问题的解由子问题的解合并而来。

两个可调参数：每次分几份（记作 k，$k = 2$ 是二分，k 很大接近桶排序）；分到什么规模停（基线规模 m，通常 1 个元素，也可以是一小段）。$k = 2$、$m = 1$ 得到归并排序；基线改成“区间短于 16 改用插入排序”就是工程上的小数组优化版。

易错：漏掉“同类”这两个字，分完之后每一层要另写一套代码，那是把任务分段，不是分治。

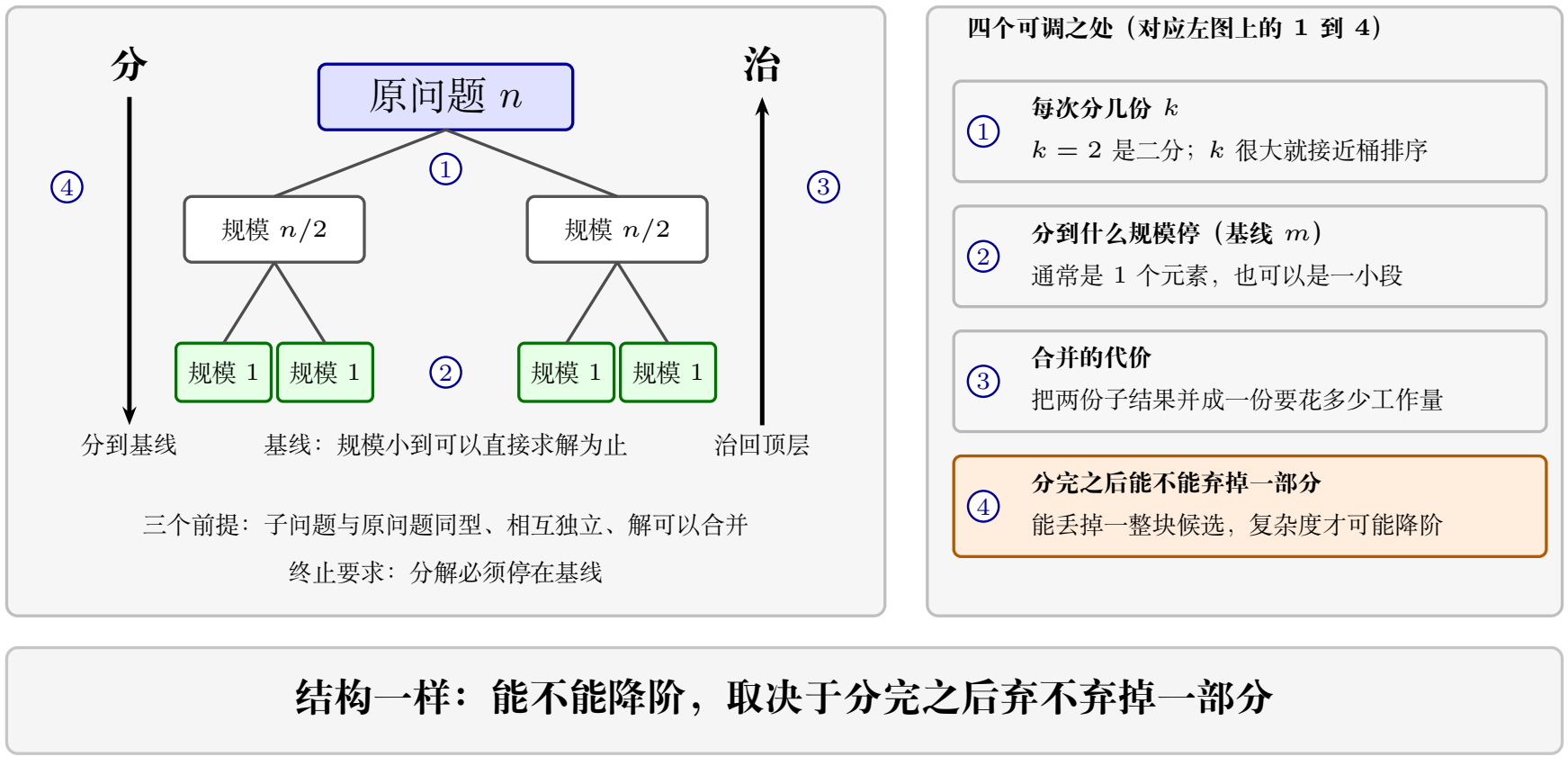


图 18: 分治的一般结构与四个可调之处

下面这段是本节的引例：分治的三要素在同一段代码里各占哪一行（Julia，完整可运行版见 `examples/appendix-dc-rec-vs-loop.jl`）：

```
1 function sort_rec!(a, lo, hi, tmp)
```

```
  if hi - lo ≥ 1                      # 基线：长度不足两个元素时什么都不做
    mid = (lo + hi) >>> 1
    sort_rec!(a, lo, mid, tmp)        # 分：两半各自同型地排
    sort_rec!(a, mid + 1, hi, tmp)
    merge_range!(a, lo, mid, hi, tmp)  # 治：把两份有序结果并成一份
  end
  return nothing
end
```

C 侧同一段骨架（`examples/appendix-dc-rec-vs-loop.c`）写法一一对应，`if (hi - lo ≥ 1)` 里 `long mid = lo + (hi - lo) / 2;` 加两次 `sortRec` 再 `mergeRange`。两种语言的计数逐个相同：分治的结构与实现语言无关，它描述的是问题的形状。

链表上的投影：拆链（九道步骤题的题五）是“分”，合链（题四）是“治”，两者都只改指针、不搬元素；链表取中点要从头走一半，代价是 $O(n)$，这条差别决定附录四的结论。

Q 13 **详解 37**

附录二：递归与循环在分治中的应用方法与差别

同一套结构的两种写法，例子仍然是归并排序，两版代码放在一起：

```
function sort_rec!(a, lo, hi, tmp)
  if hi - lo ≥ 1
    mid = (lo + hi) >>> 1
    sort_rec!(a, lo, mid, tmp)
    sort_rec!(a, mid + 1, hi, tmp)
    merge_range!(a, lo, mid, hi, tmp)
  end
  return nothing
end
```

```
function sort_iter!(a, tmp)
  n = length(a)
  width = 1
  while width < n                    # 外层循环：分的层次
    lo = 1
    while lo ≤ n                     # 内层循环：这一层要合并哪些区间
      mid = min(lo + width - 1, n)
      hi = min(lo + 2 * width - 1, n)
      if mid < hi
        merge_range!(a, lo, mid, hi, tmp)
      end
```

```
12        lo += 2 * width
13      end
14      width *= 2
15    end
16    return nothing
17  end
```

$n = 8$、输入 `4 1 3 2 8 6 7 5` 时两版合并的区间次序（`examples/appendix-dc-rec-vs-loop.jl` 实跑）：递归版 `(1,2)(3,4)(1,4)(5,6)(7,8)(5,8)(1,8)`，循环版 `(1,2)(3,4)(5,6)(7,8)(1,4)(5,8)(1,8)`。两版合并的是同一批区间、次数也相同，差在排列次序：递归版**深度优先**，循环版**按宽度一层一层**；次序不同而工作量相同，差别只在“层次的推进由谁记着”。

需要记住的状态	递归版由谁记住	循环版由谁记住
当前处在分解的第几层	调用栈里尚未返回的那些调用	外层循环的宽度变量 `width`
这一层要处理哪一段区间	函数的参数 `lo`、`hi`	由 `lo` 与 `width` 现算出的 `mid`、`hi`
子结果放在哪里	函数的返回值，向上逐层传递	临时数组与写回下标
还有哪些层没做完	调用栈中的待返回调用	循环变量自己的推进

一句话：**递归把“层次”藏在调用栈里，循环把“层次”摆在变量上**。由此带来两处代价差别，都有实跑依据：

- 栈空间：递归版深度是 $\log_2 n + 1$（实测 $n = 16$ 时 5 层、$n = 1024$ 时 11 层、$n = 4096$ 时 13 层），循环版 $O(1)$。
- 主工作量：两版都要 $n - 1$ 次合并与一份长度为 n 的临时数组；循环版省下的只是记账空间，省不下合并本身。

任何递归都可以改写成循环加显式栈（依据：“递”压栈帧存局部变量与返回位置、“归”弹栈交返回值，这两样程序自己也能保存：局部变量放进显式栈，返回位置换成“这一层接下来该做什么”的状态标记）。改写可行但不等于值得：迭代版多出来

的 `min` 收边与 `mid < hi` 判断，就是把层次边界显式写出来之后的记账负担，递归版不需要，因为调用栈本身就知道每层的区间是哪一段。

为什么迭代版要 `min` 收边：递归版的区间端点总是整齐的（分半每层都能分到准确中点）；循环版自己数步子，当 n 不是 2 的幂时，最后一组区间的右半边可能根本不存在（`mid ≥ n`），或者不满一整段（`hi > n`）。这两处不是装饰，去掉任何一处都会读越界。

尾递归的两侧实测：把求和 $S(n) = S(n-1) + n$（$S(0) = 0$）写成尾递归（递归调用是返回前最后一个动作）之后，C 侧 `clang -O2` 把它改写成循环，10^7 层仍能算完，不加优化则 10^6 层段错误（退出码 139）；Julia 侧同一函数 5×10^5 层可算完、6×10^5 层抛 `StackOverflowError`，两侧深度上限差二十倍以上。结论不是“某种语言更好”，而是**递归的栈代价不能只看算法，还要看语言的运行时与编译选项**；把“编译器会自动优化尾递归”写进代码的正确性假设里是危险的。

链表上的递归删除与递归反转，深度等于链表的节点个数（$n = 1024$ 个数据节点时约 1026 层、每层约 48 字节），归并排序的深度只有 $\log_2 n + 1$；同一个“能不能用递归”的问题，答案随分解的形状而变，这是附录三判据的来源。

详解 38

附录三：哪些场合适合递归，哪些适合循环

易错：写成“适合递归的情形”“适合循环的情形”两列清单，遇到新题目仍然不知道选哪个。下面先摆四个已经量过的例子，从例子里抽出判据，最后拿判据去回答具体题目。

四个已量过的例子：

- **例一，链表的递归删除**：结构自相似（删掉头之后剩余仍是一条链表），写成递归几乎是定义的直译；代价是每处理一个节点压一层栈，深度等于节点数（$n = 1024$ 时 1026 层，每层约 48 字节）。
- **例二，治里的线性合并**：重复次数由数据规模直接给出，写成循环是双下标扫描；改成递归后同一输入比较次数一模一样（$n = 16$ 时都 15 次、$n = 2048$ 时都 2047 次），深度却等于元素个数（每层约 96 字节）；8 万个元素仍在 8192 KiB 内，8.8 万时段错误（退出码 139）。
- **例三，归并排序的“分”**：平衡分解，深度只等于 $\log_2 n + 1$（实测 5、7、9、11、13 层），栈代价可忽略。
- **例四，快速幂**：x^n 的定义本身是递归的（算 $x^{\lfloor n/2 \rfloor}$、平方、奇数再乘一个 x），两半完全对称，深度等于折半次数加一；$n = 30$ 实测单路递归乘法 9 次、深度 6 层，把两半各算一遍的写法乘法 61 次（其中 52 次在重算同一件事）。

从这四个例子里抽出四条判据：

判据	一句话结论	反例的结论
判据一，结构	能写成“同类子问题的解加一步组合”就选递归；重复 n 次、扫描 n 个位置的线性推进选循环	把例二的线性合并写成递归：比较次数一分不省（2047 对 2047），深度从 1 涨到 2048

判据	一句话结论	反例的结论
判据二，深度	先估深度 D（平衡分解 $O(\log n)$，链状分解 $O(n)$），再估每层字节数，与限额比较；估算写成注释，不能凭感觉	链表“结构上直译、规模上危险”，被这条拦下（限额 8192 KiB ÷ 每层字节，阈值约 17 万层；实测 170002 层通过、20 万层段错误）
判据三，可读性	同阶时用来裁决：哪种写法让读者一眼看出问题的结构	用显式栈手写模拟递归：表达力一点不差，读者却要先在脑里搭一座栈
判据四，常数与开销	只在**同阶**时决定胜负；不同阶时不许用常数因子掩盖复杂度差异	把 $O(n)$ 的链表遍历改成递归：函数调用、栈、爆栈边界三处都亏

判据的适用次序：**先结构，再深度，最后在同阶时用可读性与常数裁决**。两条边界不能越过：递归不必然带来对数复杂度（附录四）；循环能表达一切递归能表达的，但表达力不等于可读性，判据三才是最后一道裁决。

Q 14 详解 39 AI 6

附录四：分治不等于 O(log N)，关键在分之后的弃

易错：把“分了治”当成对数复杂度的保证。分治只负责把问题变小，**能不能把候选丢掉**才决定代价。

例一，二分查找：能弃。 在有序数组上比较一次就能确定目标落在哪一半，另一半整块排除。实测最坏情形比较次数（`examples/appendix-search-cost.c`）：

n	二分查找比较次数	线性扫描比较次数
16	5	16
64	7	64
256	9	256
1024	11	1024
4096	13	4096
16384	15	16384

分成两半只是形式，**把一半候选真的排除掉**才是收益来源（递推 $T(n) = T(n/2) + 1$、$T(1) = 1$，n 为 2 的幂时 $T(n) = \log_2 n + 1$，与实测吻合）。

例二，求和：不能弃。 把同一集合分成 2、3、4、8 份各自求和再合并，加法次数实测全部等于 $n - 1$（递推 $T(n) = 2T(n/2) + 1$、$T(1) = 0$ 解得 $T(n) = n - 1$；分 k 份叶子仍 n 个、内部节点仍 $n - 1$）：求和没有可弃之物。

例三，快速幂：弃掉对称的那一半。 两个子问题完全相同，算一次就够，第二次是纯粹的重复劳动（$n = 30$ 单路乘法 9 次对双路 61 次）。

下面这段是二分查找的循环写法（C，`examples/appendix-search-cost.c`），每一步都在做同一件事：比较一次，然后丢掉一半：

```c
long bsearchCmps(const int* a, long n, int key, long* cmps) {
  long lo = 0, hi = n - 1, c = 0;
  while (lo ≤ hi) {
    long mid = lo + (hi - lo) / 2;
    c++;
    if (a[mid] == key) { *cmps = c; return mid; }
    if (a[mid] < key) { lo = mid + 1; }   /* 丢掉左半边 */
    else { hi = mid - 1; }                /* 丢掉右半边 */
  }
  *cmps = c;
  return -1;
}
```

把三个例子放在一起，抽象出一件事：**弃**。分治的复杂度由三个量相乘决定：每层剩下几个子问题（全留则工作量随层数累加，只剩一个则每层只付一次代价）、每层的工作量（找下一层子问题+合并）、层数（平衡分解时 $\lceil \log_2 n \rceil$）。由此得到三条判据，用来回答“一个新任务的复杂度能不能降到对数级”：

判据	一句话结论	例子
能不能弃	没有依据把一整块候选排除，复杂度就下不来	求和不能弃，无论怎么分都是 $\Theta(n)$
弃完之后每层还剩几个子问题	只剩一个才可能进入对数复杂度家族；还剩 k 个只是“分了几份”	二分查找与快速幂每轮只剩一个
每层的工作量是不是常数	不是常数的话，对数层数也救不了	链表上的二分查找

链表上的二分查找（判据三的反例）：每轮取中点、比较、决定往哪一半走，轮数确实是对数级（实测与数组二分的比较次数逐个相同：$n = 16$ 时 5、$n = 16384$ 时 15），但**走过的节点数是 $n - 1$**（递推 $\frac{n}{2} + \frac{n}{4} + \cdots + 1 = n - 1$）：链表没有随机访问，取第 k 个元素要走 k 步，所以是 $O(n)$ 而不是 $O(\log n)$。这是本周“顺序访问”那条结论的直接后果。C 与 Julia 两侧数字逐个相同，说明步数不是语法造成的，是“顺序访问”这条性质造成的。

还有一条边界：**基于猜测的、统计的、近似的算法**也可能给出 $O(\log N)$，但结果不再精确；判据一要求的“弃”是**可证的排除**，两种对数不是同一回事。

详解 40 **Q 15**

课下阅读与练习布置（两周之间的自学与练习）

课下阅读与练习布置（两周之间的自学与练习）

链表这一类结构一次课讲不完，基础与应用之间必须留出一段自己动手的时间。本讲（第 3 周，第一周）的正文与基础练习到此结束：结构与内存、基本操作、C 与 Julia 两侧的基础实现、ASEP 的链表实现（作业），另加正文之后的附录三件事，都已经过了一遍。本节只做两件事，把这段自学时间布置清楚：读什么、练什么。阅读部分只给对象与看点，不作展开讲解；练习部分只给要求与验收，不展开实现。

本节的落点在第 4 周的开头：链表的应用与复杂操作从本节布置的练习接上，第 4 周复习的起点就是这里的四道题。阅读与动手的边界也在这里写清：阅读不受已讲内容的限制，第 4 周主题的材料可以读；凡是要求动手完成的东西，只能建立在本课已讲的内容之上，因此本节四道题只用到第 1 周与本周的知识。

课下阅读：读什么，带着什么问题读

阅读材料四份，每份只给对象与看点。读法一律是带着问题读，读完对照本节末尾的三条自评。

- **阅读材料一：教材的数组与链表章。** 看点是与本讲正文的对照：数组的局限、链表的存储方式、操作代价的几条结论，两边说法不同的地方正是要停下来想的地方。带着的问题：能不能不看讲义，把链表在 C 与 Julia 两种记法下各手写一遍。
- **阅读材料二：本讲的示例代码与 `103-linked-list/examples/` 目录。** 看点是跑之外的观察：每个示例先原样跑一遍，再改一处（空值、插入位置、删除目标各改一次），看输出怎么变。带着的问题：改动落在输出的哪一行上，代价落在哪一个计数上。
- **阅读材料三：ASEP 一节里给出的模型出处综述的开头两节**（Mallick, *The Exclusion Process: A paradigm for non-equilibrium behaviour*, Physica A 418, 2015；arXiv 检索题名可得，公共 PDF）。只看模型怎么定义、排斥与不对称两条约束从哪来，**不要求读懂任何解析推导**。它是背景读物，验收在本讲的五条验证里，不在综述里。带着的问题：本讲参考实现用平行更新，综述里的连续时间模型与本讲的离散大步口径差在哪。
- **阅读材料四：第 4 周的两份预告**：链表上的综合操作题清单（合并、拆分、交换、反转、检环、相交，本讲九道步骤题的六到九就是它的预告）与 Lisp 与 Scheme 的链表记法一节（本讲 Julia 一节已用 `cons`/`nil`，下一讲展开它的来历）。只要求知道下一周讲什么，不设任何动手要求，读不懂的地方留到第 4 周。

三条自评。 四份材料读完之后逐条对照，三条都能说清才算读完这一轮。

1. 能说清节点（存放元素的存储单元，由数据域与指针域两部分组成）、头节点（链表中排在最前面的节点）、尾节点（链表中最后一个节点，它的指针域为空）与空值两种记法（C 侧记 `NULL`，Julia 侧记 `nothing`）的定义。

2. 能说清链表在哪一类操作上比数组便宜、在哪一类操作上比数组贵，并把各自的前提一起说出来：链表的插入与删除以已经持有前驱位置为前提，数组的 $O(1)$ 访问以有序号为前提。
3. 能说清 ASEP（Asymmetric Simple Exclusion Process，不对称简单排斥过程：一条一维格点链加若干粒子，每个格点最多容纳一个粒子，粒子按两个不相等的概率向两个方向尝试跃迁，只有目标格点为空时移动才成功）里的排斥与不对称两个词各指什么，以及周期性边界（链的两端相接，粒子从一端走出就从另一端出现）是什么意思。

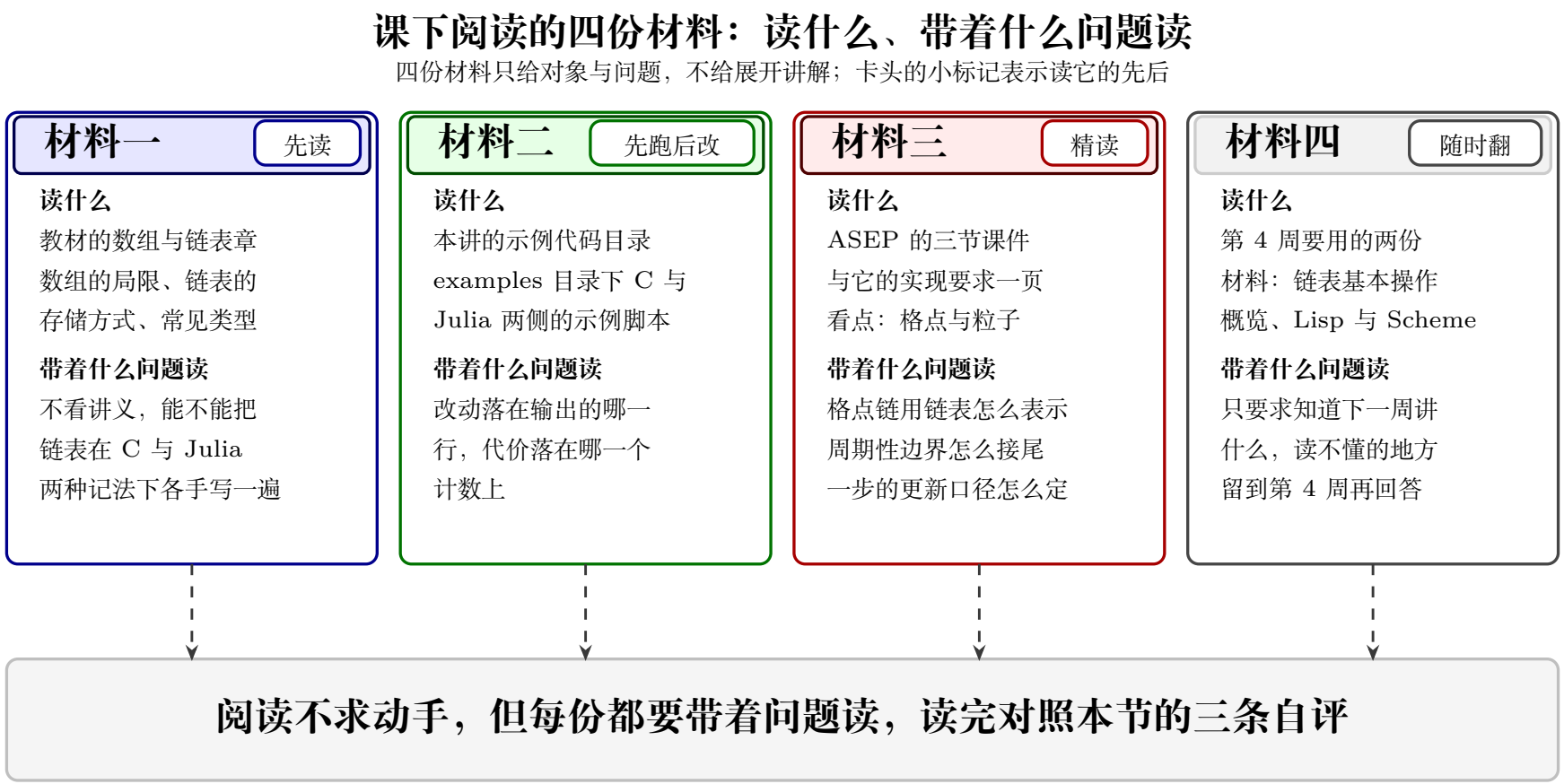


图 19: 课下阅读四份材料的对象与问题

Q 16 AI 7

课下练习与作业：题目、提交与验收要求

本节四道题：题一至题三动手，题四只提问。语言统一用 C 与 Julia，工具链统一为 clang 与 lldb。课下四题与上一节当堂做的三题分工不同：课堂三题当堂完成，本节四题课下完成，其中**题目二（ASEP 的链表实现）是本讲作业的核心题**，其实现要求六条、验证方式五条与交付物清单全部写在 ASEP 一节的“作业布置”里，以那里为准。

- **题目一（同一个结构的两种记法）：** 用 C 与 Julia 各写一遍单链表的创建、插入、删除与打印，要求能编译运行并给出输出。
- **题目二（ASEP 的链表实现，本周核心作业）：** 按 ASEP 一节“实现要求六条”用链表（不使用数组）实现一维 ASEP，按“验证方式五条”在 README 里逐条给出期望值、实测值与结论，附轨迹数据文件与绘图脚本；先做 ASEP 一节练习二第一问的纸面四答再动手。完整题干、交付物、验收与口径要求见那一节。
- **题目三（链表与一维数组的比较，短答）：** 从四个方面比较两种结构：存储结构、内存分配、访问时间的步骤、插入与删除的步骤。要求写出步骤本身，不停在复杂度结论上。
- **题目四（思考题，只提问）：** 是否可能把链表与一维数组的优点结合起来。本周只提出问题，不给答案，答案在第 4 周讲静态链表（用下标模拟指针的链表变体）一节。

提交要求。 按课程 Git 流程五步：

1. fork 教师为本次作业创建的仓库，仓库名不改；学生身份由账号决定，不由仓库名决定。
2. 克隆到本地：`git clone <自己的 fork 仓库地址>`。
3. 在本地完成作业，用 git 记录过程，提交信息写清这一次改的是什么。
4. 推送前先 `git pull origin main` 同步教师仓库的更新，作业说明与示例代码可能被修正，然后 push。
5. 评分以截止时间前的最后一次推送为准，截止时间之后推送扣分；截止时间以课程平台公告为准。

只上传可运行的源代码与说明文档（轨迹数据落成文本文件，属于允许上传的文档类型），不上传大图片、二进制与数据文件；评阅只看 Git 仓库内的记录。

验收要求。 四条判据，四道题通用。

1. 可独立运行：C 侧在 `clang -Wall -Wextra -g` 下到零告警，Julia 侧能直接跑通。
2. 可复现：同一份代码与同一个随机数种子给出同一串结果，README 说明结构与运行方法。
3. 三种边界必须处理：空链表、只有一个节点的链表、删第一个数据节点与删尾节点（后两种合成一条边界，都落在头尾的处理上）。
4. 四道题只用到第 1 周与本周已讲的知识，答案里不出现未讲过的主题。

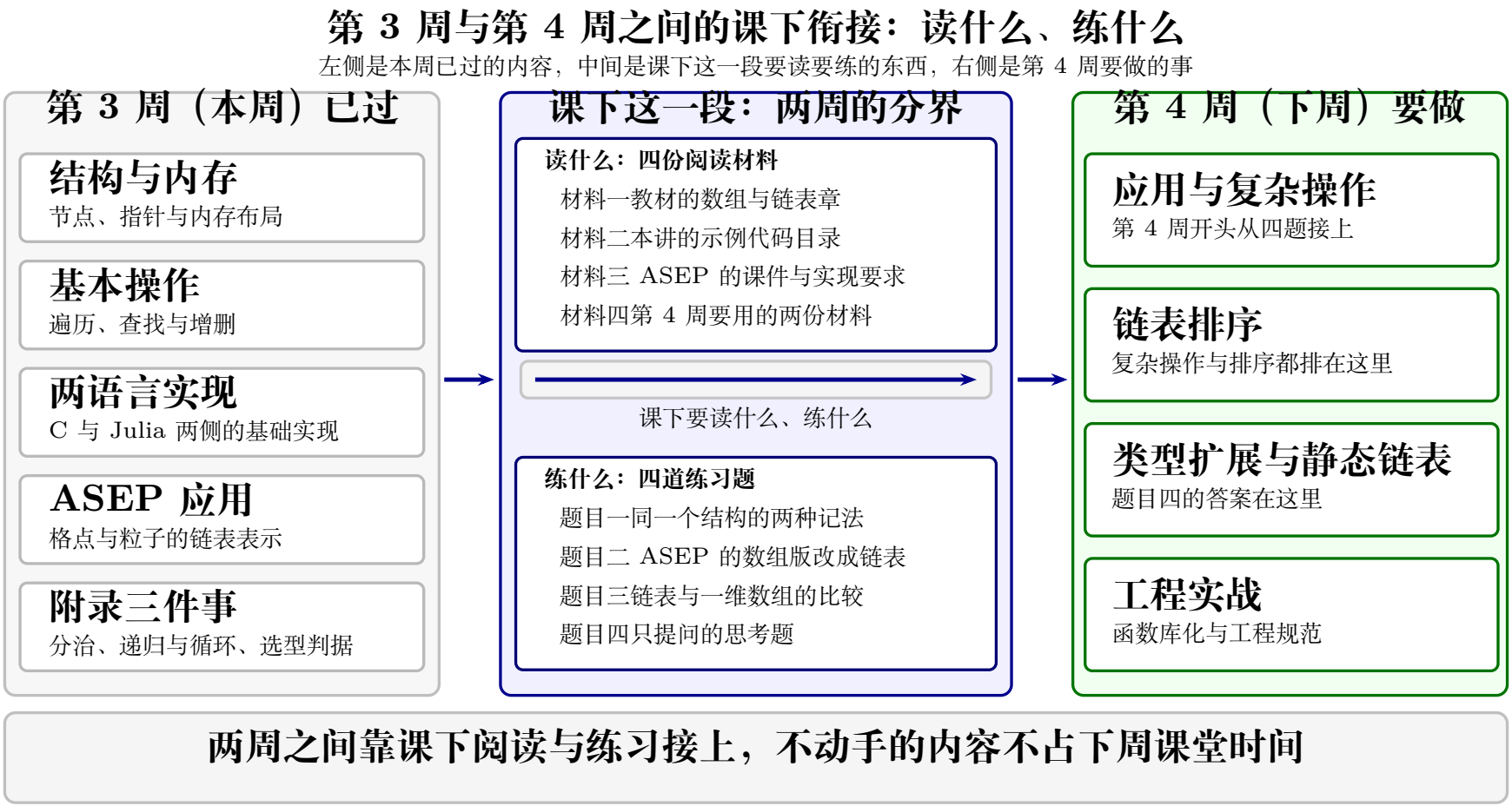


图 20: 两周之间的课下衔接：读什么、练什么

Q 17 AI 8 详解 41 详解 42